

The Socket Exchange

Experiment Report

COL 334 – Computer Networks
Assignment 2

Implementation language	C (POSIX sockets, FreeBSD libc)
Concurrency / I/O model	Single-threaded, non-blocking, <code>kqueue/kevent</code>
Platform	FreeBSD 14 (amd64) virtual machine, host <code>ragnarok</code>
Transport	TCP over the loopback interface <code>lo0</code> , <code>127.0.0.1:5000</code>
Harness	<code>python3 experiment.py <n></code> (provided with the assignment)

This report documents Experiments 1–8 of Section 6 of the assignment handout. For every experiment it states the question, the investigation approach, the exact FreeBSD commands and tools used, the raw evidence (terminal screenshots and captured output), and the answer supported by that evidence. Section 2 describes the concurrency and I/O design of the Exchange Server, which several of the experiments probe directly.

Contents

1	Experimental Setup	2
2	Implementation Decisions	2
2.1	Concurrency and I/O approach	2
2.2	Why this approach	3
2.3	Other design decisions relevant to TCP behaviour	3
3	Experiment 1 – Listening and Connected Sockets	4
4	Experiment 2 – Observing TCP Connection States	6
5	Experiment 3 – TCP as a Byte Stream	9
6	Experiment 4 – One Client Should Not Stall the Others	12
7	Experiment 5 – Multiple Clients and I/O Multiplexing	14
8	Experiment 6 – FIN vs. RST: Orderly and Abrupt Termination	19
9	Experiment 7 – Backpressure and the Slow Receiver	23
10	Experiment 8 – Unexpected Client Disconnection	26
11	Summary of Tools Used	30
A	Bonus – Connection Scalability and I/O Design	30
A.1	Setup and tooling	30
A.2	Changes made to the Exchange Server	31
A.3	Client-side source-IP rotation (for $N > 55\,000$)	32
A.4	Full measurement table	32
A.5	Screenshots	33
A.6	Answers to Section 6.9 questions	38
A.7	Notes and caveats	40

1 Experimental Setup

All processes – the Exchange Server, the harness-generated clients and the observation tools – run inside a single FreeBSD virtual machine. The server binds `127.0.0.1:5000`, so every connection in this report is a loopback connection and all packet captures are taken on `lo0`.

Launchers. The harness always starts the server through the standard launcher, so it never needs to know the implementation language:

```
$ cat server/run-server
#!/bin/sh
exec "$(dirname "$0")/../exchange_server" "$@"

$ cat client/run-trader
#!/bin/sh
exec "$(dirname "$0")/../trader_client" "$@"

$ cat client/run-market-data
#!/bin/sh
exec "$(dirname "$0")/../market_data_client" "$@"
```

Running an experiment. Each experiment is started from the submission root with

```
$ python3 experiment.py <experiment-number>
```

and observed from one or more additional SSH sessions into the same VM while the harness is in its observation phase.

Notation in the command listings. Throughout this report a line beginning with `$` is a command that was run; a line beginning with `#` and followed by prose is a comment added here for the reader, not part of the command. The packet captures (`tcpdump`), the syscall traces (`ktrace`) and the kernel tuning (`sysctl`) all require privilege and were run as `root`; the screenshots therefore show a `#` shell prompt for those sessions.

A note on the harness's readiness probe. Before every experiment the harness opens a short-lived TCP connection to check that the server is listening and then closes it abortively (`SO_LINGER` with a zero timeout). This probe is visible in several packet captures as a connection that is answered with an `RST`, and in the server's syscall traces as `accept()` returning `errno 53` (`ECONNABORTED`). It is not part of any experiment; it is pointed out where it appears so that it is not mistaken for experimental traffic.

2 Implementation Decisions

2.1 Concurrency and I/O approach

The Exchange Server is a **single process with a single thread** that runs one **event loop built on FreeBSD's `kqueue`/`kevent`** interface. There is no thread and no process per connection. Concretely:

- The listening socket (`fd 3`) is created with `socket()`, `setsockopt(SO_REUSEADDR)`, `bind()` and `listen()`, and is registered on the `kqueue` (`fd 4`) with an `EVFILT_READ` filter.
- **Every** socket – listening and accepted – is put into non-blocking mode with `fcntl(fd, F_SETFL, O_NONBLOCK)`. The server therefore never blocks inside `accept()`, `read()` or `write()`; the only blocking call in the whole program is `kevent()`.

- Each accepted socket is registered with *two* filters, `EVFILT_READ` and `EVFILT_WRITE`, both with the `EV_CLEAR` flag, i.e. edge-triggered notification.
- Because the notification is edge-triggered, `accept()` is called in a loop until it returns `EAGAIN`, so a burst of simultaneous connections is drained in one pass. `ECONNABORTED` is tolerated and simply skipped (this is what happens to the harness’s abortive readiness probe).

A symbol scan of the server binary agrees with the traces: it imports `kqueue` and `kevent` and contains *no* reference to `poll`, `select` or any `pthread` function. The event-loop structure is directly visible in the `ktrace/kdump` evidence used in Experiments 4–8; for example the `accept` path in Experiment 5:

```
17538 exchange_server CALL kevent(0x4,0,0,0x820883df0,0x40,0)
17538 exchange_server STRU struct kevent[] = { { ident=3, filter=EVFILT_READ, ..., data=0x5, ... } }
17538 exchange_server RET kevent 1
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:64427 }
17538 exchange_server RET accept 5
17538 exchange_server CALL fcntl(0x5,F_SETFL,0x6<0_RDWR|0_NONBLOCK>)
17538 exchange_server CALL kevent(0x4,0x820883cc0,0x2,0,0,0)
17538 exchange_server STRU struct kevent[] = { { ident=5, filter=EVFILT_READ, flags=0x21<EV_ADD|EV_CLEAR>,
... }
                                { ident=5, filter=EVFILT_WRITE, flags=0x21<EV_ADD|EV_CLEAR>,
... } }
...
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server RET accept -1 errno 35 Resource temporarily unavailable
```

2.2 Why this approach

- **No client can stall another.** With one thread per connection, or with blocking I/O on a single thread, a client that opens a connection and then says nothing would park the server inside `recv()`. With `kevent()` the server only ever touches a descriptor that the kernel has already declared ready, so an idle or half-speaking client costs nothing. This is exactly what Experiment 4 measures.
- **Cost per connection is a few hundred bytes, not a stack.** A thread-per-connection design pays a full thread stack (typically hundreds of kilobytes of virtual memory) per client. Here a connection is one entry in a `realloc()`-grown table plus its I/O buffers. Experiment 7 shows the server’s RSS staying at exactly 2180 KiB while 5000 market-data updates are pushed to four clients.
- **`kqueue` is the native, scalable mechanism on FreeBSD.** Unlike `select()` and `poll()`, it does not rescan the whole descriptor set on every call, and unlike `select()` it has no `FD_SETSIZE` limit – which matters for the bonus scalability target.

2.3 Other design decisions relevant to TCP behaviour

Message framing. The protocol is line-oriented, and TCP is a byte stream, so every connection owns a growable input buffer. After each `read()` the buffer is scanned for `'\n'`; each complete line found is dispatched, and any trailing partial line is compacted to the front of the buffer and kept until more bytes arrive. The server therefore makes no assumption about how the sender split its `send()` calls: one message may arrive in four reads (Experiment 3) and several messages may arrive in one read.

Output buffering and back-pressure. Writes are non-blocking. If a `write()` to a client is short or returns `EAGAIN`, the remainder is appended to that connection’s own output buffer and the `EVFILT_WRITE` event – which is registered for every connection at `accept` time – drains it later. A

slow reader can therefore only make *its own* output buffer grow; the event loop keeps servicing everybody else. This is what Experiment 7 verifies.

Roles. A connection starts out with no role. The first command it sends decides it: `LOGIN` makes it a Trader Client, `SUBSCRIBE` makes it a Market-Data Client. Afterwards, commands that do not belong to the role are rejected with `ERROR`, which is how the capability matrix of Section 2.8 of the handout is enforced.

Disconnect handling. A peer that closes gives `EVFILT_READ` with `EV_EOF` and `read()` returning 0. The server then deletes both filters for that descriptor, releases the connection slot (cancelling nothing else) and calls `close()`. One client disappearing never touches another client’s state.

Graceful shutdown. `SIGINT/SIGTERM` interrupt the blocking `kevent()` with `EINTR`; the loop then closes every connection slot, the `kqueue` and the listening socket before exiting, which is visible at the end of every `kdump` trace in this report.

3 Experiment 1 – Listening and Connected Sockets

After the client has connected, identify the TCP sockets associated with the Exchange Server. How does the listening socket differ from the socket representing the connection to the client?

Approach and tools

The harness (`python3 experiment.py 1`) starts the server and leaves exactly one established, idle client connection in place. While it waits, the server’s sockets are enumerated from two independent angles: `sockstat`, which maps sockets to the owning process and file descriptor, and `netstat`, which shows the TCP state machine’s view.

```
$ sockstat -4 -l          # listening IPv4 sockets
$ sockstat -4 -c          # connected IPv4 sockets
$ sockstat -4 -l | grep -E 'USER|exchange'  # same, filtered to the server
$ sockstat -4 -c | grep -E 'USER|exchange|python'
$ netstat -an -p tcp | grep 5000
```

Observations

```

zfs list -o name mypool/usr

More columns and their order can be defined by separating them with commas:

zfs list -o mountpoint,name,avail

-- Benedict Reuschling <bcr@FreeBSD.org>
[harsh@ragnarok ~]$ cd compiled
[harsh@ragnarok ~/compiled]$ sockstat -4 -l
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
harsh     exchange_s 1874 3   tcp4   127.0.0.1:5000  *:.*
harsh     node      1002 14  tcp4   127.0.0.1:36880 *:.*
harsh     node      964 14  tcp4   127.0.0.1:45866 *:.*
root      sshd      813 7   tcp4   *:22           *:.*
root      syslogd   715 7   udp4   *:514          *:.*
[harsh@ragnarok ~/compiled]$ sockstat -4 -c
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
harsh     exchange_s 1874 5   tcp4   127.0.0.1:5000  127.0.0.1:47532
harsh     python3.12 1873 3   tcp4   127.0.0.1:47532 127.0.0.1:5000
harsh     sshd-sessi 1387 7   tcp4   10.0.2.15:22    10.0.2.2:54901
root      sshd-sessi 1382 7   tcp4   10.0.2.15:22    10.0.2.2:54901
harsh     sshd-sessi 1290 7   tcp4   10.0.2.15:22    10.0.2.2:58990
root      sshd-sessi 1283 7   tcp4   10.0.2.15:22    10.0.2.2:58990
harsh     node      1186 15  tcp4   127.0.0.1:36880 127.0.0.1:30236
harsh     node      1186 19  tcp4   10.0.2.15:29100 54.82.101.0:443
harsh     sshd-sessi 1044 7   tcp4   10.0.2.15:22    10.0.2.2:53578
harsh     sshd-sessi 1044 8   tcp4   127.0.0.1:30236 127.0.0.1:36880
harsh     sshd-sessi 1044 10  tcp4   127.0.0.1:38851 127.0.0.1:36880
root      sshd-sessi 1038 7   tcp4   10.0.2.15:22    10.0.2.2:53578
harsh     node      1002 15  tcp4   127.0.0.1:36880 127.0.0.1:38851
[harsh@ragnarok ~/compiled]$

```

Figure 1: `sockstat -4 -l` and `sockstat -4 -c`: the full listening and connected IPv4 socket tables while the experiment client is connected. The Exchange Server is PID 1874.

```

[harsh@ragnarok ~/compiled]$
[harsh@ragnarok ~/compiled]$
[harsh@ragnarok ~/compiled]$ sockstat -4 -l | grep -E 'USER|exchange'
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
harsh     exchange_s 1874 3   tcp4   127.0.0.1:5000  *:.*
[harsh@ragnarok ~/compiled]$ sockstat -4 -c | grep -E 'USER|exchange|python'
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
harsh     exchange_s 1874 5   tcp4   127.0.0.1:5000  127.0.0.1:47532
harsh     python3.12 1873 3   tcp4   127.0.0.1:47532 127.0.0.1:5000
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      127.0.0.1.47532    ESTABLISHED
tcp4      0      0 127.0.0.1.47532     127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                LISTEN
[harsh@ragnarok ~/compiled]$

```

Figure 2: The same information filtered to the server and its client, followed by `netstat -an -p tcp | grep 5000`: two ESTABLISHED rows (one per endpoint of the loopback connection) and one LISTEN row.

```

--- from: sockstat -4 -l | grep -E 'USER|exchange' -----
USER  COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
harsh exchange_s 1874 3   tcp4   127.0.0.1:5000  *:.*          <- listening

--- from: sockstat -4 -c | grep -E 'USER|exchange|python' -----
USER  COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
harsh exchange_s 1874 5   tcp4   127.0.0.1:5000  127.0.0.1:47532 <- connected
harsh python3.12 1873 3   tcp4   127.0.0.1:47532 127.0.0.1:5000  <- client end

--- from: netstat -an -p tcp | grep 5000 -----

```

```

tcp4  0  0  127.0.0.1.5000    127.0.0.1.47532    ESTABLISHED
tcp4  0  0  127.0.0.1.47532  127.0.0.1.5000     ESTABLISHED
tcp4  0  0  127.0.0.1.5000    *.*                 LISTEN

```

The Exchange Server (PID 1874) owns exactly two TCP sockets:

- **file descriptor 3**, local address 127.0.0.1:5000, foreign address *:*, state LISTEN;
- **file descriptor 5**, local address 127.0.0.1:5000, foreign address 127.0.0.1:47532, state ESTABLISHED.

The client process (`python3.12`, PID 1873) holds the mirror image of the second socket, which is why `netstat` lists the connection twice: on loopback both endpoints live on the same machine.

The gap in the numbering – the first client gets `fd 5`, not `fd 4` – is not an accident: `fd 4` is the `kqueue` the server waits on, which is a descriptor but not a socket and therefore never appears in `sockstat` or `netstat`. It is visible in the descriptor table captured in Experiment 5 (Figure 12).

Answer. The server has one *listening* socket (`fd 3`) and one *connected* socket (`fd 5`). The listening socket is only a half association – local endpoint 127.0.0.1:5000 with no peer (**) and state LISTEN – and it carries no data; its only role is to be handed to `accept()`. The connected socket is identified by the full four-tuple 127.0.0.1:5000 ↔ 127.0.0.1:47532, is in state ESTABLISHED and is the descriptor on which the server actually reads and writes. `accept()` creates a brand-new descriptor for every client while the listening socket keeps listening on the same port.

4 Experiment 2 – Observing TCP Connection States

How does the TCP state of the client–server connection change during its lifetime, and what events cause the observed changes?

Approach and tools

The harness (`python3 experiment.py 2`) connects a client, holds it idle for 10 seconds (phase 1), closes it, and keeps the server alive for another 15 seconds (phase 2). Two observers run in parallel: repeated `netstat` snapshots for the state column, taken by hand at each announced phase of the run, and `tcpdump` on `lo0` for the packets that cause the transitions. The harness prints **Phase 1** and **Phase 2** as it reaches them, which is what the snapshots were timed against; because the two phases last 10 and 15 seconds there is no need to poll faster.

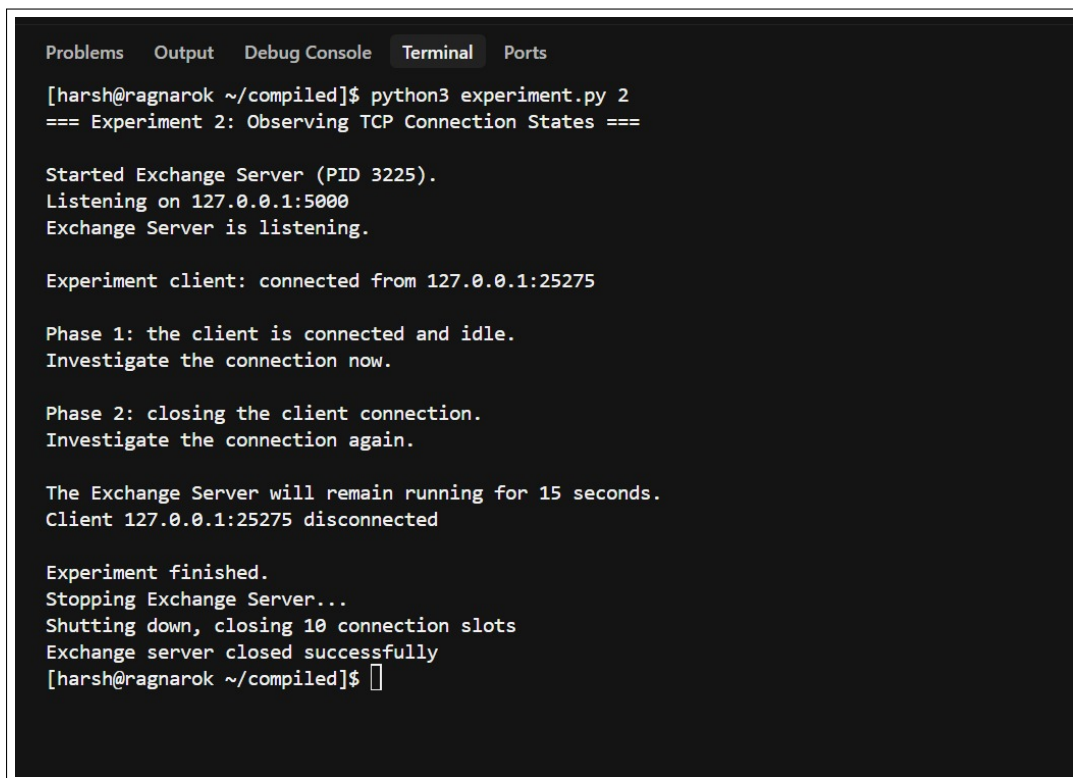
```

# terminal 2 -- the same command, re-run at each stage of the run:
#   twice during phase 1 (client connected and idle),
#   then repeatedly after phase 2 closed the connection,
#   and once more after the server itself exited
$ netstat -an -p tcp | grep 5000

# terminal 3 -- the packets that drive the state machine
$ tcpdump -i lo0 -n -S tcp port 5000

```

Observations



```

Problems Output Debug Console Terminal Ports

[harsh@ragnarok ~/compiled]$ python3 experiment.py 2
=== Experiment 2: Observing TCP Connection States ===

Started Exchange Server (PID 3225).
Listening on 127.0.0.1:5000
Exchange Server is listening.

Experiment client: connected from 127.0.0.1:25275

Phase 1: the client is connected and idle.
Investigate the connection now.

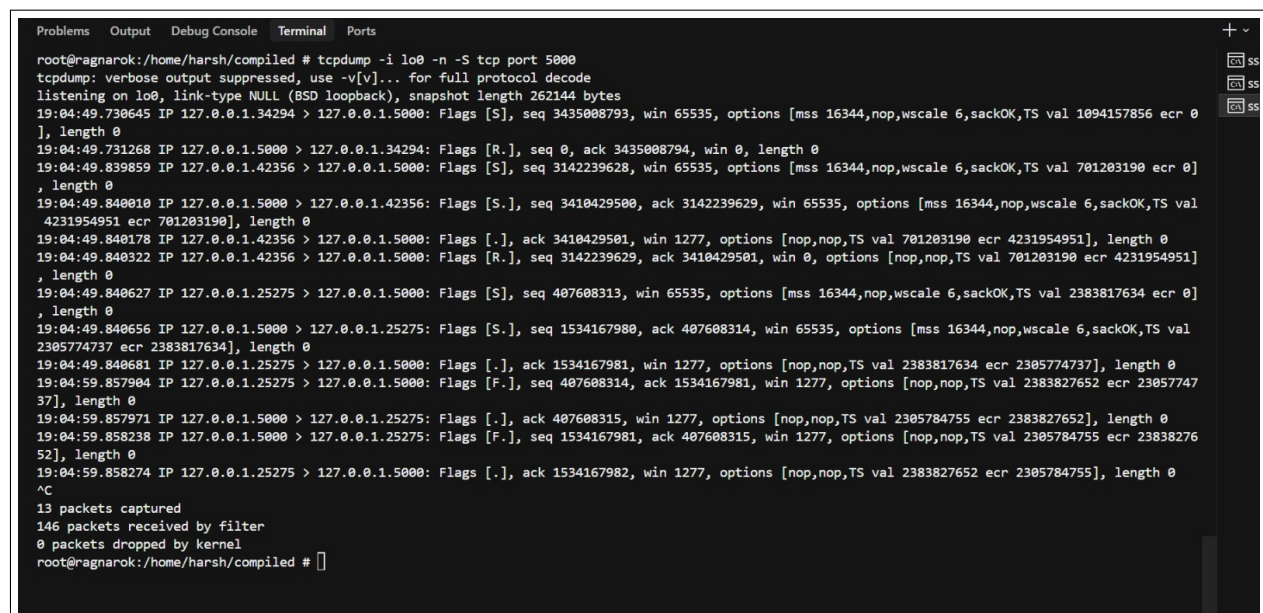
Phase 2: closing the client connection.
Investigate the connection again.

The Exchange Server will remain running for 15 seconds.
Client 127.0.0.1:25275 disconnected

Experiment finished.
Stopping Exchange Server...
Shutting down, closing 10 connection slots
Exchange server closed successfully
[harsh@ragnarok ~/compiled]$

```

Figure 3: Harness output. Server PID 3225; the experiment client connects from 127.0.0.1:25275, stays idle for phase 1, and is closed at the start of phase 2.



```

Problems Output Debug Console Terminal Ports

root@ragnarok:/home/harsh/compiled # tcpdump -i lo0 -n -S tcp port 5000
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
19:04:49.730645 IP 127.0.0.1.34294 > 127.0.0.1.5000: Flags [S], seq 3435008793, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 1094157856 ecr 0], length 0
19:04:49.731268 IP 127.0.0.1.5000 > 127.0.0.1.34294: Flags [R.], seq 0, ack 3435008794, win 0, length 0
19:04:49.839859 IP 127.0.0.1.42356 > 127.0.0.1.5000: Flags [S], seq 3142239628, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 701203190 ecr 0], length 0
19:04:49.840010 IP 127.0.0.1.5000 > 127.0.0.1.42356: Flags [S.], seq 3410429500, ack 3142239629, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 4231954951 ecr 701203190], length 0
19:04:49.840178 IP 127.0.0.1.42356 > 127.0.0.1.5000: Flags [.], ack 3410429501, win 1277, options [nop,nop,TS val 701203190 ecr 4231954951], length 0
19:04:49.840322 IP 127.0.0.1.42356 > 127.0.0.1.5000: Flags [R.], seq 3142239629, ack 3410429501, win 0, options [nop,nop,TS val 701203190 ecr 4231954951], length 0
19:04:49.840627 IP 127.0.0.1.25275 > 127.0.0.1.5000: Flags [S], seq 407608313, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 2383817634 ecr 0], length 0
19:04:49.840656 IP 127.0.0.1.5000 > 127.0.0.1.25275: Flags [S.], seq 1534167980, ack 407608314, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 2305774737 ecr 2383817634], length 0
19:04:49.840681 IP 127.0.0.1.25275 > 127.0.0.1.5000: Flags [.], ack 1534167981, win 1277, options [nop,nop,TS val 2383817634 ecr 2305774737], length 0
19:04:59.857904 IP 127.0.0.1.25275 > 127.0.0.1.5000: Flags [F.], seq 407608314, ack 1534167981, win 1277, options [nop,nop,TS val 2383827652 ecr 2305774737], length 0
19:04:59.857971 IP 127.0.0.1.5000 > 127.0.0.1.25275: Flags [.], ack 407608315, win 1277, options [nop,nop,TS val 2385784755 ecr 2383827652], length 0
19:04:59.858238 IP 127.0.0.1.5000 > 127.0.0.1.25275: Flags [F.], seq 1534167981, ack 407608315, win 1277, options [nop,nop,TS val 2385784755 ecr 2383827652], length 0
19:04:59.858274 IP 127.0.0.1.25275 > 127.0.0.1.5000: Flags [.], ack 1534167982, win 1277, options [nop,nop,TS val 2383827652 ecr 2305784755], length 0
^C
13 packets captured
146 packets received by filter
0 packets dropped by kernel
root@ragnarok:/home/harsh/compiled #

```

Figure 4: tcpdump -i lo0 -n -S tcp port 5000: the three-way handshake at 19:04:49, ten seconds of complete silence, then the four-way teardown at 19:04:59.

```

[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4          0      0 127.0.0.1.5000      127.0.0.1.25275    ESTABLISHED
tcp4          0      0 127.0.0.1.25275     127.0.0.1.5000     ESTABLISHED
tcp4          0      0 127.0.0.1.5000      *.*                LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4          0      0 127.0.0.1.5000      127.0.0.1.25275    ESTABLISHED
tcp4          0      0 127.0.0.1.25275     127.0.0.1.5000     ESTABLISHED
tcp4          0      0 127.0.0.1.5000      *.*                LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4          0      0 127.0.0.1.5000      *.*                LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4          0      0 127.0.0.1.5000      *.*                LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4          0      0 127.0.0.1.5000      *.*                LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
[harsh@ragnarok ~/compiled]$

```

Figure 5: `netstat -an -p tcp | grep 5000`, re-run by hand at each stage – eight invocations in all. The first produces no output because it was run before the harness had started the server; the next two are taken during phase 1 and show both endpoints `ESTABLISHED` alongside the `LISTEN` socket; the following four are taken after phase 2 closed the connection and show only `LISTEN`; the last one, after the server exited, is empty again.

The capture in Figure 4 contains the whole life of the connection (client port 25275):

```

19:04:49.840627 127.0.0.1.25275 > 127.0.0.1.5000 Flags [S], seq 407608313
19:04:49.840656 127.0.0.1.5000 > 127.0.0.1.25275 Flags [S.], seq 1534167980, ack 407608314
19:04:49.840681 127.0.0.1.25275 > 127.0.0.1.5000 Flags [.] , ack 1534167981
----- 10 s of phase 1: not a single packet -----
19:04:59.857904 127.0.0.1.25275 > 127.0.0.1.5000 Flags [F.], seq 407608314
19:04:59.857971 127.0.0.1.5000 > 127.0.0.1.25275 Flags [.] , ack 407608315
19:04:59.858238 127.0.0.1.5000 > 127.0.0.1.25275 Flags [F.], seq 1534167981
19:04:59.858274 127.0.0.1.25275 > 127.0.0.1.5000 Flags [.] , ack 1534167982

```

Mapping packets to states:

Event on the wire	Client state	Server state
before anything	CLOSED	LISTEN (fd 3)
SYN sent	SYN_SENT	SYN_RCVD
SYN,ACK + final ACK	ESTABLISHED	ESTABLISHED (new fd)
idle phase (no packets)	ESTABLISHED	ESTABLISHED
client <code>close()</code> sends FIN	FIN_WAIT_1	CLOSE_WAIT
server ACKs the FIN	FIN_WAIT_2	CLOSE_WAIT
server sees EOF, <code>close()</code> sends FIN	TIME_WAIT [†]	LAST_ACK
client ACKs	(entry removed)	CLOSED

[†] Skipped on this system – see the second bullet below.

Two points are worth noting about what `netstat` actually shows.

- **The idle phase produces no packets at all.** A TCP connection that is established but unused is pure kernel state; nothing is sent to keep it alive (keep-alives are off by default). This is why an idle client is invisible on the wire yet still fully `ESTABLISHED` in `netstat`.

- **No TIME_WAIT row appears after the close.** The connection simply disappears and only the LISTEN socket remains, even though the client performed the active close and should therefore have entered TIME_WAIT for 2MSL. This is a loopback-specific FreeBSD behaviour: the `sysctl net.inet.tcp.nolocaltimewait` makes an endpoint whose peer address is local skip the TIME_WAIT state entirely, and on the FreeBSD 14 series it still defaults to 1 (it was flipped to 0 and deprecated only in later FreeBSD, for RFC 9293 compliance). The value on this VM can be confirmed with `sysctl net.inet.tcp.nolocaltimewait`. Experiment 6 shows the same connection reported as CLOSED rather than TIME_WAIT while its descriptor is still open, which is the other half of the same behaviour.

The two exchanges at 19:04:49.730 and 19:04:49.839 in Figure 4 are the harness's readiness probes and not part of the experiment. They fail in two different ways, which is worth reading carefully: the first (port 34294) is answered by the server's host with an RST because nothing was listening on port 5000 yet – a refused connection that never existed – whereas the second (port 42356) completes the handshake and is then torn down by an RST sent *by the probe itself*, which is the harness's deliberate SO_LINGER-zero close. Only the second one tells the harness that the server is up, after which it starts the experiment proper.

Answer. The connection walks the standard TCP state machine, and every transition is caused by a packet or a system call, never by time. `connect()` on the client and the handshake `SYN → SYN,ACK → ACK` take both ends from CLOSED/LISTEN to ESTABLISHED; the connection then stays ESTABLISHED through ten completely silent seconds; the client's `close()` emits a FIN, putting the client in FIN_WAIT_1 and the server in CLOSE_WAIT; the server's `read()` returns 0, it calls `close()` and emits its own FIN (LAST_ACK); the client's final ACK closes both ends. Only the LISTEN socket survives.

5 Experiment 3 – TCP as a Byte Stream

Does the server receive the message as one complete unit, or in multiple pieces? Explain what this demonstrates about TCP vs. application-level message boundaries.

Approach and tools

The harness sends the single protocol message "LOGIN experiment_trader\n" as four separate `send()` calls – "LOGIN ", "experiment", "_trader", "\n" – with a 200 ms pause between them. The traffic is captured with payloads printed in ASCII so that the split is visible byte for byte:

```
$ tcpdump -i lo0 -n -A tcp port 5000
```

Observations

```

root@ragnarok:/home/harsh/compiled # tcpdump -i lo0 -n -A tcp port 5000
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
03:45:40.919723 IP 127.0.0.1.63659 > 127.0.0.1.5000: Flags [S], seq 1165544018, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 4079201088 ecr 0
], length 0
E..<..@.....Ex.R.....0...?.....
.#.@....
03:45:40.919922 IP 127.0.0.1.5000 > 127.0.0.1.63659: Flags [R.], seq 0, ack 1165544019, win 0, length 0
E..(.@.....Ex.SP.....
03:45:41.028499 IP 127.0.0.1.57740 > 127.0.0.1.5000: Flags [S], seq 2004805460, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 2223285339 ecr 0
], length 0
E..<..@.....W~.T.....0...?.....
...[....
03:45:41.028553 IP 127.0.0.1.5000 > 127.0.0.1.57740: Flags [S.], seq 402732527, ack 2004805461, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val
3056887093 ecr 2223285339], length 0
E..<..@.....5.W~.U.....0...?.....
.4e5...[
03:45:41.028604 IP 127.0.0.1.57740 > 127.0.0.1.5000: Flags [.], ack 1, win 1277, options [nop,nop,TS val 2223285339 ecr 3056887093], length 0
E..4..@.....W~.U..5.....(.....
...[.4e5
03:45:41.028773 IP 127.0.0.1.57740 > 127.0.0.1.5000: Flags [R.], seq 1, ack 1, win 0, options [nop,nop,TS val 2223285339 ecr 3056887093], length 0
E..4..@.....W~.U..5.....(.....
...[.4e5
03:45:41.029183 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [S], seq 2738259465, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 571047032 ecr 0]
, length 0
E..<..@.....?@...6. ....0...?.....
" |x....
03:45:41.029323 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [S.], seq 1564807594, ack 2738259466, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val
1872050621 ecr 571047032], length 0
E..<..@.....?@]E...6.
....0...?.....
o.9." |x
03:45:41.029370 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [.], ack 1, win 1277, options [nop,nop,TS val 571047032 ecr 1872050621], length 0
E..4..@.....?@...6.
]E.....(.....
" |xo.9.
03:45:41.029576 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 1:7, ack 1, win 1277, options [nop,nop,TS val 571047032 ecr 1872050621], length 6
E.....@.....?@...6.

```

Figure 6: Start of the capture: the two readiness probes – port 63659 refused with an RST from the server’s host because the server was not listening yet, then port 57740 connecting successfully and aborting itself with an RST – and finally the experiment connection from 127.0.0.1:16192 completing its handshake.

```

E..4..@.....?@...6.
]E.....(.....
"
|xo.9.
03:45:41.029576 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 1:7, ack 1, win 1277, options [nop,nop,TS val 571047032 ecr 1872050621], length 6
E..4..@.....?@...6.
]E.....(.....
"
|xo.9.LOGIN
03:45:41.070223 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [.], ack 7, win 1277, options [nop,nop,TS val 1872050651 ecr 571047032], length 0
E..4..@.....?@...6.....(.....
o.9."
|x
03:45:41.243816 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 7:17, ack 1, win 1277, options [nop,nop,TS val 571047248 ecr 1872050651], length 10
E..>..@.....?@...6..]E.....2.....
"
)Po.9.experiment
03:45:41.278234 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [.], ack 17, win 1277, options [nop,nop,TS val 1872050871 ecr 571047248], length 0
E..4..@.....?@...6.....(.....
o.:."
)P
03:45:41.456138 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 17:24, ack 1, win 1277, options [nop,nop,TS val 571047453 ecr 1872050871], length 7
E.;.;..@.....?@...6..]E...../.....
"
~.o.:..trader
03:45:41.490000 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [.], ack 24, win 1277, options [nop,nop,TS val 1872051071 ecr 571047453], length 0
E..4..@.....?@...6..!.....(.....
o.;;"
~.
03:45:41.671446 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 24:25, ack 1, win 1277, options [nop,nop,TS val 571047673 ecr 1872051071], length 1
E..5..@.....?@...6..!E.....).....
"
~.o.;.
"
03:45:41.672246 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [P.], seq 1:4, ack 25, win 1277, options [nop,nop,TS val 1872051262 ecr 571047673], length 3
E..7..@.....?@...6..".....+.....
o.<.>"
~.OK
"
03:45:41.710056 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [.], ack 4, win 1277, options [nop,nop,TS val 571047714 ecr 1872051262], length 0
E..4..@.....?@...6.."]E.....(.....
"
~.o.<.>
^C
18 packets captured
661 packets received by filter
0 packets dropped by kernel
root@ragnarok:/home/harsh/compiled #

```

Figure 7: The four fragments of one application message, each in its own TCP segment. The first three are acknowledged by a pure ACK; the fourth – the one carrying the newline – is acknowledged on the server’s OK reply itself (Flags [P.], seq 1:4, ack 25), which is the first byte the server sends in the whole exchange.

03:45:41.029576	16192 > 5000	Flags [P.], seq 1:7, length 6	...LOGIN
03:45:41.070223	5000 > 16192	Flags [.], ack 7, length 0	
03:45:41.243816	16192 > 5000	Flags [P.], seq 7:17, length 10	...experiment
03:45:41.278234	5000 > 16192	Flags [.], ack 17, length 0	
03:45:41.456138	16192 > 5000	Flags [P.], seq 17:24, length 7	..._trader
03:45:41.490000	5000 > 16192	Flags [.], ack 24, length 0	
03:45:41.671446	16192 > 5000	Flags [P.], seq 24:25, length 1	...(newline)
03:45:41.672246	5000 > 16192	Flags [P.], seq 1:4, length 3	...OK
03:45:41.710056	16192 > 5000	Flags [.], ack 4, length 0	

The evidence has two independent parts:

1. **The message crosses the network in four pieces.** Four data segments of 6, 10, 7 and 1 bytes carry one 24-byte application message. Their sequence numbers are contiguous (1:7, 7:17, 17:24, 24:25) – TCP is delivering a byte stream and has no idea that these 24 bytes form one unit. Each one is delivered to the server’s socket as soon as it arrives, so the server’s `read()` returns four times.
2. **The server nevertheless answers exactly once.** OK is sent at 03:45:41.672246, 0.8 ms after the fourth segment but roughly 640 ms after the first. The server buffered the first three fragments, found no ‘\n’, and only dispatched `LOGIN` `experiment` `trader` when the newline finally arrived. The acknowledgement pattern says the same thing from the other side: the first three segments draw a bare ACK and nothing else, while the fourth is acknowledged on the OK segment itself (ack 25 riding on the 3-byte reply) – the server had nothing to say until the delimiter completed the message.

Answer. The server receives the message in *multiple* pieces – four segments of 6, 10, 7 and 1 bytes, the first three drawing a bare ACK and the fourth being acknowledged on the reply itself – and it still produces exactly one OK. This is the essential point about TCP: it is a byte

stream with no message boundaries, so a sender's `send()` calls have no relationship to the receiver's `recv()` calls. Application-level framing must be reconstructed by the receiver, which our server does by buffering bytes per connection and splitting them at the `'\n'` delimiter.

6 Experiment 4 – One Client Should Not Stall the Others

When Client 1 is idle, can the server still accept and service Client 2? Identify the server operation determining the answer.

Approach and tools

The harness makes Client 1 send a *deliberately incomplete* message – "LOGIN blocked_client" with no trailing newline – and then go permanently silent. This is the harsher version of an idle client: the server has already received bytes for a message that will never be completed. Two seconds later Client 2 connects and sends a complete message, and the harness times how long the reply takes.

The decisive question is *which syscall the server is sitting in*, so the process itself is inspected rather than only the network:

```
$ sockstat -4 -c | grep -E 'USER|exchange'
$ netstat -an -p tcp | grep 5000
$ procstat -k $(pgrep -f exchange_server)      # kernel stack: what is it blocked on?
$ ps -H -o pid,tid,%cpu,wchan,comm -p $(pgrep -f exchange_server)
```

Observations

```
Problems Output Debug Console Terminal Ports
[harsh@ragnarok ~/compiled]$
[harsh@ragnarok ~/compiled]$
[harsh@ragnarok ~/compiled]$ python3 experiment.py 4
=== Experiment 4: One Client Should Not Stall the Others ===

Started Exchange Server (PID 1925).
Listening on 127.0.0.1:5000
Exchange Server is listening.

Client 1: connected from 127.0.0.1:61838

Client 1 will send an incomplete application message.
Client 1 will now remain silent.

Client 2: connected from 127.0.0.1:58885
Client 2 will send a complete application message.
6: LOGIN active_client
User active_client logged in from 127.0.0.1:58885

Client 2 response: 'OK'
Elapsed time: 0.005 seconds

Investigate the behavior of both TCP connections.
Press Ctrl-C when you are finished.
█
```

Figure 8: Harness output. Client 1 (port 61838) sends a partial message and falls silent; Client 2 (port 58885) then logs in and is answered 'OK' in **0.005 seconds**. The server's own log line 6: `LOGIN active_client` names the descriptor it served, and no such line ever appears for Client 1.

```

[harsh@ragnarok ~/compiled]$
[harsh@ragnarok ~/compiled]$
[harsh@ragnarok ~/compiled]$
[harsh@ragnarok ~/compiled]$ sockstat -4 -c | grep -E 'USER|exchange'
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
harsh     exchange_s 1925 5    tcp4   127.0.0.1:5000  127.0.0.1:61838
harsh     exchange_s 1925 6    tcp4   127.0.0.1:5000  127.0.0.1:58885
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      0      0  127.0.0.1.5000      127.0.0.1.58885      ESTABLISHED
tcp4      0      0  127.0.0.1.58885     127.0.0.1.5000      ESTABLISHED
tcp4      0      0  127.0.0.1.5000      127.0.0.1.61838     ESTABLISHED
tcp4      0      0  127.0.0.1.61838     127.0.0.1.5000      ESTABLISHED
tcp4      0      0  127.0.0.1.5000      *.*                  LISTEN
[harsh@ragnarok ~/compiled]$ procstat -k $(pgrep -f exchange_server)
PID      TID  COMM          TDNAME          KSTACK
1925 100328 exchange_server -                mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_
kevent_generic sys_kevent amd64_syscall fast_syscall_common
[harsh@ragnarok ~/compiled]$ ps -H -o pid,tid,%cpu,wchan,comm -p $(pgrep -f exchange_server)
PID      LWP  %CPU  WCHAN          COMMAND
1925 100328 0.0  kqread exchange_server
[harsh@ragnarok ~/compiled]$

```

Figure 9: System-level evidence: both connections are ESTABLISHED and owned by the server (fds 5 and 6); `procstat -k` shows the single server thread parked in `kqueue_scan/kqueue_kevent`; and `ps -H` reports one thread with `WCHAN = kqread`.

USER	COMMAND	PID	FD	PROTO	LOCAL ADDRESS	FOREIGN ADDRESS	
harsh	exchange_s	1925	5	tcp4	127.0.0.1:5000	127.0.0.1:61838	<- silent client 1
harsh	exchange_s	1925	6	tcp4	127.0.0.1:5000	127.0.0.1:58885	<- client 2

Recv-Q Send-Q					
tcp4	0	0	127.0.0.1.5000	127.0.0.1.58885	ESTABLISHED
tcp4	0	0	127.0.0.1.58885	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	127.0.0.1.61838	ESTABLISHED
tcp4	0	0	127.0.0.1.61838	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	*.*	LISTEN

PID	TID	COMM	TDNAME	KSTACK
1925	100328	exchange_server	-	mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_kevent_generic sys_kevent amd64_syscall ...

PID	LWP	%CPU	WCHAN	COMMAND
1925	100328	0.0	kqread	exchange_server

Three facts fall out of this:

- Client 2's round trip took **5 milliseconds** – there is no stall of any kind. Both clients are simultaneously ESTABLISHED and both are owned by the same single-threaded server process (fds 5 and 6).
- The server has **exactly one thread** (TID 100328), so it cannot be hiding the problem by dedicating a thread to the silent client.
- That one thread is blocked in `kqueue_scan` with `WCHAN = kqread`, i.e. inside `kevent()` – *not* inside `recv()` on fd 5. Client 1's incomplete message is sitting in the server's per-connection input buffer waiting for a newline that will never come, and that costs nothing because the server never asks the kernel for more bytes from fd 5 until the kernel says fd 5 is readable again.

Two smaller details in the same figures confirm where Client 1's bytes ended up. First, every Recv-Q in the `netstat` output is 0, including fd 5's: the kernel is holding nothing back, so the server has already *read* the partial "LOGIN blocked_client" and is holding it in its own input buffer – the wait is happening in user space, on a delimiter, not in a syscall. Second, the server's log in Figure 8 prints 6: `LOGIN active_client` for Client 2 and no corresponding line for fd 5, which is the same fact seen from the application side: the partial line was buffered but never dispatched as a command.

Answer. Yes – Client 2 is accepted and answered OK in 0.005 s while Client 1 sits silently mid-message. The operation that decides this is the **blocking call at the top of the event loop: kevent() on the kqueue**, not a per-connection `recv()`. `procstat -k` and `ps` both show the server's only thread sleeping in `kqueue_scan/kqread`, so it wakes up for whichever descriptor becomes ready and reads only descriptors the kernel has already reported as readable. Had the server called a blocking `recv()` on Client 1's socket, its kernel stack would instead show `soreceive`, and Client 2's LOGIN would have gone unanswered until the harness gave up – it waits 5 seconds for the reply and would have reported `None` instead of `'OK'`.

7 Experiment 5 – Multiple Clients and I/O Multiplexing

At a particular point, which client connections are ready for the server to service, and what evidence supports this?

Approach and tools

The harness opens five simultaneous connections and then has only clients 1, 3 and 5 send an application message; clients 2 and 4 stay idle. Readiness is a property of the kernel's view of each socket, so it is captured from two directions:

```
# 1. record every syscall the server makes, including the kevent return values
$ rm -f /tmp/exp5.ktr
$ while :; do pid=$(pgrep -n exchange_server); if [ -n "$pid" ]; then \
    ktrace -f /tmp/exp5.ktr -t cit -p $pid; echo "attached to $pid"; break; fi; done
$ ktrace -C                                # stop tracing when done

# 2. read the trace back
$ kdump -f /tmp/exp5.ktr | grep -B2 -A6 kevent | less
$ kdump -f /tmp/exp5.ktr | grep -B6 'LOGIN client'
$ kdump -f /tmp/exp5.ktr | grep -E 'ident=[0-9]+, filter=EVFILT_READ, flags=0x20<EV_CLEAR>'

# 3. map the server's descriptors back to real sockets, and see queued bytes
$ PID=$(pgrep -n exchange_server); procstat -f $PID; ps -o pid,stat,wchan,comm -p $PID
$ netstat -an -p tcp | grep 5000

# 4. second run: freeze the server the instant it starts listening, so that the
#     bytes the clients send stay queued in the kernel and netstat can show them
$ while :; do pid=$(pgrep -n exchange_server); \
    if [ -n "$pid" ] && sockstat -4 -l | grep -q ':5000'; then \
        kill -STOP $pid; echo "stopped $pid"; break; fi; done
$ kill -CONT $(pgrep -n exchange_server)
```

Observations

```

Started Exchange Server (PID 17538).
Listening on 127.0.0.1:5000
Exchange Server is listening.

Creating five simultaneous TCP connections...

Client 1: connected from 127.0.0.1:64427
Client 2: connected from 127.0.0.1:28067
Client 3: connected from 127.0.0.1:43792
Client 4: connected from 127.0.0.1:65363
Client 5: connected from 127.0.0.1:38472

Clients 1, 3, and 5 will send application messages.
Clients 2 and 4 will remain idle.

Client 1: sent application data.
5: LOGIN client_1
User client_1 logged in from 127.0.0.1:64427
Client 3: sent application data.
7: LOGIN client_3
User client_3 logged in from 127.0.0.1:43792
Client 5: sent application data.
9: LOGIN client_5
User client_5 logged in from 127.0.0.1:38472

The experiment is now in the observation phase.
Investigate which connections were active.
Press Ctrl-C when you are finished.
Stopping Exchange Server...
Client 127.0.0.1:64427 (user client_1) disconnected
Client 127.0.0.1:28067 disconnected
Client 127.0.0.1:43792 (user client_3) disconnected
Client 127.0.0.1:65363 disconnected
Client 127.0.0.1:38472 (user client_5) disconnected
Shutting down, closing 10 connection slots
Exchange server closed successfully
[harsh@ragnarok ~/compiled]$

```

Figure 10: Harness output for the traced run (server PID 17538). Clients 1, 3 and 5 connect from ports 64427, 43792 and 38472 and send LOGIN; clients 2 and 4 (ports 28067 and 65363) stay idle. The server logs each login against the descriptor number: 5:, 7:, 9:.

```

Problems Output Debug Console Terminal Ports
[harsh@ragnarok ~/compiled]$ rm -f /tmp/exp5.ktr
while :; do pid=$(pgrep -n exchange_server); if [ -n "$pid" ]; then ktrace -f /tmp/exp5.ktr -t cit -p $pid; echo "attached to $pid"; break; fi; done
attached to 17538
[harsh@ragnarok ~/compiled]$ ktrace -C
kdump -f /tmp/exp5.ktr | grep -B2 -A6 kevent | less
17538 exchange_server STRU struct kevent[] = { { ident=3, filter=EVFILT_READ, flags=0, fflags=0, data=0x1, udata=0x0 } }
17538 exchange_server RET kevent 1
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server RET accept -1 errno 53 Software caused connection abort
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server RET accept -1 errno 35 Resource temporarily unavailable
17538 exchange_server CALL kevent(0x4,0,0,0x820883df0,0x40,0)
17538 exchange_server STRU struct kevent[] = { }
17538 exchange_server STRU struct kevent[] = { { ident=3, filter=EVFILT_READ, flags=0, fflags=0, data=0x5, udata=0x0 } }
17538 exchange_server RET kevent 1
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:64427 }
17538 exchange_server RET accept 5
17538 exchange_server CALL fcntl(0x5,F_GETFL)
17538 exchange_server RET fcntl 6
17538 exchange_server CALL fcntl(0x5,F_SETFL,0x6<O_RDWR|O_NONBLOCK>)
17538 exchange_server RET fcntl 0
17538 exchange_server CALL kevent(0x4,0x820883cc0,0x2,0,0,0)
17538 exchange_server STRU struct kevent[] = { { ident=5, filter=EVFILT_READ, flags=0x21<EV_ADD|EV_CLEAR>, fflags=0, data=0, udata=0x0 } }
{ ident=5, filter=EVFILT_WRITE, flags=0x21<EV_ADD|EV_CLEAR>, fflags=0, data=0, udata=0x0 } }
17538 exchange_server STRU struct kevent[] = { }
17538 exchange_server RET kevent 0
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:28067 }
17538 exchange_server RET accept 6
17538 exchange_server CALL fcntl(0x6,F_GETFL)
17538 exchange_server RET fcntl 6
17538 exchange_server CALL fcntl(0x6,F_SETFL,0x6<O_RDWR|O_NONBLOCK>)
17538 exchange_server RET fcntl 0
17538 exchange_server CALL kevent(0x4,0x820883cc0,0x2,0,0,0)
17538 exchange_server STRU struct kevent[] = { { ident=6, filter=EVFILT_READ, flags=0x21<EV_ADD|EV_CLEAR>, fflags=0, data=0, udata=0x0 } }
{ ident=6, filter=EVFILT_WRITE, flags=0x21<EV_ADD|EV_CLEAR>, fflags=0, data=0, udata=0x0 } }
17538 exchange_server STRU struct kevent[] = { }

```

Figure 11: Attaching `ktrace` to the server as soon as it starts, and the beginning of the trace: five `accept()` calls returning fds 5–9, each immediately set `O_NONBLOCK` and registered on the kqueue with `EVFILT_READ` and `EVFILT_WRITE`.

```

Problems Output Debug Console Terminal Ports
tcp4 0 0 127.0.0.1:5000 *. * LISTEN
[harsh@ragnarok ~/compiled]$ kill -CONT $(pgrep -n exchange_server); netstat -an -p tcp | grep 5000
kill: usage: kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... or kill -l [sigspec]
[harsh@ragnarok ~/compiled]$ PID=$(pgrep -n exchange_server); procstat -f $PID; ps -o pid,stat,wchan,comm -p $PID
PID COMM FD T V FLAGS REF OFFSET PRO NAME
17538 exchange_server trace v r rw----- - - - /tmp/exp5.ktr
17538 exchange_server text v r r----- - - - /home/harsh/compiled/exchange_server
17538 exchange_server cwd v d r----- - - - /home/harsh/compiled
17538 exchange_server root v d r----- - - - /
17538 exchange_server 0 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 1 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 2 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 3 s - rw---n-- 2 0 TCP 0 0 127.0.0.1:5000 *:0
17538 exchange_server 4 k - rw----- 2 0 - -
17538 exchange_server 5 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:64427
17538 exchange_server 6 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:28067
17538 exchange_server 7 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:43792
17538 exchange_server 8 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:65363
17538 exchange_server 9 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:38472
PID STAT WCHAN COMMAND
17538 Ss kqread exchange_server
[harsh@ragnarok ~/compiled]$ PID=$(pgrep -n exchange_server); procstat -f $PID; ps -o pid,stat,wchan,comm -p $PID
PID COMM FD T V FLAGS REF OFFSET PRO NAME
17538 exchange_server trace v r rw----- - - - /tmp/exp5.ktr
17538 exchange_server text v r r----- - - - /home/harsh/compiled/exchange_server
17538 exchange_server cwd v d r----- - - - /home/harsh/compiled
17538 exchange_server root v d r----- - - - /
17538 exchange_server 0 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 1 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 2 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 3 s - rw---n-- 2 0 TCP 0 0 127.0.0.1:5000 *:0
17538 exchange_server 4 k - rw----- 2 0 - -
17538 exchange_server 5 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:64427
17538 exchange_server 6 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:28067
17538 exchange_server 7 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:43792
17538 exchange_server 8 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:65363
17538 exchange_server 9 s - rw---n-- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:38472
PID STAT WCHAN COMMAND

```

Figure 12: `procstat -f` for the server: fd 3 is the listening socket 127.0.0.1:5000 *:0, fd 4 is the kqueue, and fds 5–9 are the five client connections in the same order the harness created them. `ps` shows the process in `kqread`.

```

Problems Output Debug Console Terminal Ports
17538 exchange_server GIO fd 5 read 15 bytes
"LOGIN client_1
"
17538 exchange_server RET read 15/0xf
17538 exchange_server CALL write(0x1,0x3bbdd409000,0x12)
17538 exchange_server GIO fd 1 wrote 18 bytes
"5: LOGIN client_1
"
17538 exchange_server CALL kevent(0x4,0,0,0x82083df0,0x40,0)
17538 exchange_server STRU struct kevent[] = { }
17538 exchange_server STRU struct kevent[] = { { ident=7, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
17538 exchange_server RET kevent 1
17538 exchange_server CALL read(0x7,0x3bbdd4183c1,0xa0)
17538 exchange_server GIO fd 7 read 15 bytes
"LOGIN client_3
"
17538 exchange_server RET read 15/0xf
17538 exchange_server CALL write(0x1,0x3bbdd409000,0x12)
17538 exchange_server GIO fd 1 wrote 18 bytes
"7: LOGIN client_3
"
17538 exchange_server CALL kevent(0x4,0,0,0x82083df0,0x40,0)
17538 exchange_server STRU struct kevent[] = { }
17538 exchange_server STRU struct kevent[] = { { ident=9, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
17538 exchange_server RET kevent 1
17538 exchange_server CALL read(0x9,0x3bbdd4186c1,0xa0)
17538 exchange_server GIO fd 9 read 15 bytes
"LOGIN client_5
"
17538 exchange_server RET read 15/0xf
17538 exchange_server CALL write(0x1,0x3bbdd409000,0x12)
17538 exchange_server GIO fd 1 wrote 18 bytes
"9: LOGIN client_5
"
[harsh@ragnarok ~/compiled]$ kdump -f /tmp/exp5.ktr | grep -E 'ident=[0-9]+, filter=EVFILT_READ, flags=0x20<EV_CLEAR>'
17538 exchange_server STRU struct kevent[] = { { ident=5, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
17538 exchange_server STRU struct kevent[] = { { ident=7, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
17538 exchange_server STRU struct kevent[] = { { ident=9, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
[harsh@ragnarok ~/compiled]$

```

Figure 13: The decisive evidence: `kevent()` returns `EVFILT_READ` readiness for `ident=5`, `ident=7` and `ident=9` only, each with `data=0xf` (15 bytes pending), and the server reads exactly those descriptors. The final `grep` lists all three ready descriptors together.

The descriptor-to-client mapping from `procstat -f` (Figure 12) is unambiguous:

fd	Client	Peer port	Sent data?
3	(listening socket)	*:0	—
4	(kqueue)	—	—
5	Client 1	64427	yes
6	Client 2	28067	no (idle)
7	Client 3	43792	yes
8	Client 4	65363	no (idle)
9	Client 5	38472	yes

and the kqueue reports readiness for precisely fds 5, 7 and 9:

```

17538 exchange_server STRU struct kevent[] = { { ident=5, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, data=0
xf, ... }
                                { ident=5, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, data=0
                                xbf88, ... }
                                { ident=6, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, data=0
                                xbf88, ... }
                                { ident=7, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, data=0
                                xbf88, ... }
                                { ident=8, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, data=0
                                xbf88, ... }
                                { ident=9, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, data=0
                                xbf88, ... } }
17538 exchange_server RET kevent 6
17538 exchange_server CALL read(0x5,0x3bbbdd4180c1,0xa0)
17538 exchange_server GIO fd 5 read 15 bytes
"LOGIN client_1
"
17538 exchange_server RET read 15/0xf
17538 exchange_server CALL write(0x1,0x3bbbdd409000,0x12)
17538 exchange_server GIO fd 1 wrote 18 bytes
"5: LOGIN client_1
"
17538 exchange_server RET write 18/0x12
17538 exchange_server CALL write(0x1,0x3bbbdd409000,0x2d)
17538 exchange_server GIO fd 1 wrote 45 bytes
"User client_1 logged in from 127.0.0.1:64427
"
17538 exchange_server RET write 45/0x2d
17538 exchange_server CALL write(0x5,0x20105b,0x3)
17538 exchange_server GIO fd 5 wrote 3 bytes
"OK
"
17538 exchange_server RET write 3
17538 exchange_server CALL read(0x5,0x3bbbdd4180c1,0xa0)
17538 exchange_server RET read -1 errno 35 Resource temporarily unavailable
... (identically for ident=7 and ident=9; fds 6 and 8 are never read)

```

That first return needs reading carefully, because it carries six events, not three. Exactly *one* of them is an `EVFILT_READ` event, on `ident=5`; the other five are `EVFILT_WRITE` events, one for every connection including the two idle ones. This is not a contradiction – it is what write-readiness means. A socket is “ready to write” whenever its send buffer has room, and all five send buffers are empty, which is why each reports `data=0xbf88` (49 032 bytes of free space). Write-readiness therefore says nothing about whether a client has spoken; only `EVFILT_READ` does, and that filter fires for descriptors 5, 7 and 9 alone. The targeted `grep` in Figure 13 makes the point without the noise: across the whole trace, exactly three lines match `filter=EVFILT_READ, flags=0x20<EV_CLEAR>`, and they are `ident=5, 7 and 9`.

`data=0xf` is the kernel telling the application how many bytes are waiting: 15, exactly the length of `"LOGIN client_1\n"`. The two intervening writes are to `fd 1`, the server’s own standard output – they are the log lines visible in Figure 10 and carry no network traffic; the only socket write is the 3-byte `"OK\n"` on `fd 5`. Note also the trailing `read()` returning `EAGAIN`: because the filters are edge-triggered (`EV_CLEAR`), the server keeps reading a ready descriptor until the kernel says there is nothing left.

The same picture is visible from outside the process. In a second run the server was stopped

with `kill -STOP` the moment it began listening, so it never consumed anything: the bytes the three active clients sent stayed queued in the kernel receive buffers, where `netstat` can show them. (This is also why the harness afterwards reports that the server had to be killed – a stopped process cannot handle `SIGTERM`.) Worth being precise about what those sockets are: the server was frozen before it ever called `accept()`, so the five connections exist only in the kernel's accept queue and no server descriptor points at them yet. The three-way handshake and the receive buffering are done entirely by the TCP stack, which is exactly why the byte counts are still visible even though the application is not running.

```
[harsh@ragnarok ~/compiled]$  
[harsh@ragnarok ~/compiled]$  
[harsh@ragnarok ~/compiled]$ python3 experiment.py 5  
=== Experiment 5: Multiple Clients and I/O Multiplexing (Optional) ===  
  
Started Exchange Server (PID 9147).  
Listening on 127.0.0.1:5000  
Exchange Server is listening.  
  
Creating five simultaneous TCP connections...  
  
Client 1: connected from 127.0.0.1:42196  
Client 2: connected from 127.0.0.1:62300  
Client 3: connected from 127.0.0.1:13037  
Client 4: connected from 127.0.0.1:26978  
Client 5: connected from 127.0.0.1:53133  
  
Clients 1, 3, and 5 will send application messages.  
Clients 2 and 4 will remain idle.  
  
Client 1: sent application data.  
Client 3: sent application data.  
Client 5: sent application data.  
  
The experiment is now in the observation phase.  
Investigate which connections were active.  
Press Ctrl-C when you are finished.  
Stopping Exchange Server...  
Exchange Server did not terminate; killing it.  
[harsh@ragnarok ~/compiled]$
```

Figure 14: The second run (server PID 9147): client 1 = 42196, client 2 = 62300, client 3 = 13037, client 4 = 26978, client 5 = 53133.

```

[harsh@ragnarok ~/compiled]$
[harsh@ragnarok ~/compiled]$
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      15      0  127.0.0.1.5000      127.0.0.1.53133      ESTABLISHED
tcp4      0      0  127.0.0.1.53133      127.0.0.1.5000      ESTABLISHED
tcp4      0      0  127.0.0.1.5000      127.0.0.1.26978      ESTABLISHED
tcp4      0      0  127.0.0.1.26978      127.0.0.1.5000      ESTABLISHED
tcp4      15      0  127.0.0.1.5000      127.0.0.1.13037      ESTABLISHED
tcp4      0      0  127.0.0.1.13037      127.0.0.1.5000      ESTABLISHED
tcp4      0      0  127.0.0.1.5000      127.0.0.1.62300      ESTABLISHED
tcp4      0      0  127.0.0.1.62300      127.0.0.1.5000      ESTABLISHED
tcp4      15      0  127.0.0.1.5000      127.0.0.1.42196      ESTABLISHED
tcp4      0      0  127.0.0.1.42196      127.0.0.1.5000      ESTABLISHED
tcp4      0      0  127.0.0.1.5000      127.0.0.1.59941      CLOSED
tcp4      0      0  127.0.0.1.5000      *.*                  LISTEN
[harsh@ragnarok ~/compiled]$

```

Figure 15: `netstat -an -p tcp`: the server-side rows for ports 53133, 13037 and 42196 (clients 5, 3 and 1) carry `Recv-Q = 15`; the rows for the idle clients 62300 and 26978 carry `Recv-Q = 0`. The `CLOSED` row is the leftover of the harness's readiness probe.

Answer. At the moment the three active clients have spoken, exactly **three** of the five connections are ready: the ones belonging to clients 1, 3 and 5. Two independent pieces of evidence agree. (i) `kevent()` returns `EVFILT_READ` events only for `ident=5, 7 and 9`, each carrying `data=0xf` (15 bytes pending), and `procstat -f` maps those descriptors to peer ports 64427, 43792 and 38472 – clients 1, 3 and 5. (ii) `netstat` shows `Recv-Q = 15` on exactly those three server-side sockets and `Recv-Q = 0` on the two idle ones. The idle connections 2 and 4 are perfectly healthy `ESTABLISHED` sockets; they are simply never reported *readable*, so the server never touches them and pays nothing for them. (They are reported *writable*, because their send buffers are empty and therefore always have room – but write-readiness says nothing about whether a client has sent anything.)

8 Experiment 6 – FIN vs. RST: Orderly and Abrupt Termination

How does abrupt termination differ from orderly shutdown? Identify the TCP event on the network and the corresponding socket behaviour.

Approach and tools

The harness runs two parts back to back on the same server. In **Part A** a client connects and calls `shutdown(SHUT_WR)`, the orderly close. In **Part B** a second client sets `SO_LINGER` with a zero timeout and calls `close()`, which makes the kernel abort the connection instead of closing it. Three observers run simultaneously:

```

# packets
$ tcpdump -i lo0 -n -tttt -S tcp port 5000

# socket state, once per second with timestamps
$ while ;; do date +%H:%M:%S; netstat -an -p tcp | grep 5000; echo ---; sleep 1; done

# what the server's syscalls see
$ rm -f /tmp/exp6.ktr
$ while ;; do pid=$(pgrep -n exchange_server); if [ -n "$pid" ]; then \

```

```

ktrace -f /tmp/exp6.ktr -t cit -p $pid; break; fi; done
$ kdump -f /tmp/exp6.ktr | grep -n -E 'EV_EOF|errno 54|ECONNRESET|CALL +close'
$ kdump -f /tmp/exp6.ktr | grep -A6 'EV_EOF'
$ kdump -f /tmp/exp6.ktr | grep -n -E 'RET +accept'

```

Observations

```

root@ragnarok:/home/harsh/compiled # tcpdump -i lo0 -n -tttt -S tcp port 5000
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
2026-09-08 15:11:11.480199 IP 127.0.0.1.23897 > 127.0.0.1.5000: Flags [S], seq 3215378964, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 56364
4169 ecr 0], length 0
2026-09-08 15:11:11.480553 IP 127.0.0.1.5000 > 127.0.0.1.23897: Flags [R.], seq 0, ack 3215378965, win 0, length 0
2026-09-08 15:11:11.588739 IP 127.0.0.1.56301 > 127.0.0.1.5000: Flags [S], seq 4201627391, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 27492
92607 ecr 0], length 0
2026-09-08 15:11:11.590037 IP 127.0.0.1.5000 > 127.0.0.1.56301: Flags [S.], seq 3093378638, ack 4201627392, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 649293137 ecr 2749292607], length 0
2026-09-08 15:11:11.590111 IP 127.0.0.1.56301 > 127.0.0.1.5000: Flags [.], ack 3093378639, win 1277, options [nop,nop,TS val 2749292618 ecr 649293137], length 0
2026-09-08 15:11:11.590489 IP 127.0.0.1.56301 > 127.0.0.1.5000: Flags [R.], seq 4201627392, ack 3093378639, win 0, options [nop,nop,TS val 2749292618 ecr 649293137], length 0
2026-09-08 15:11:11.593527 IP 127.0.0.1.56436 > 127.0.0.1.5000: Flags [S], seq 751935905, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3326117646 ecr 0], length 0
2026-09-08 15:11:11.593600 IP 127.0.0.1.5000 > 127.0.0.1.56436: Flags [S.], seq 3975613210, ack 751935906, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 798935012 ecr 3326117646], length 0
2026-09-08 15:11:11.593676 IP 127.0.0.1.56436 > 127.0.0.1.5000: Flags [.], ack 3975613211, win 1277, options [nop,nop,TS val 3326117646 ecr 798935012], length 0
2026-09-08 15:11:11.593872 IP 127.0.0.1.56436 > 127.0.0.1.5000: Flags [F.], seq 751935906, ack 3975613211, win 1277, options [nop,nop,TS val 3326117646 ecr 798935012], length 0
2026-09-08 15:11:11.593937 IP 127.0.0.1.5000 > 127.0.0.1.56436: Flags [.], ack 751935907, win 1277, options [nop,nop,TS val 798935012 ecr 3326117646], length 0
2026-09-08 15:11:11.607184 IP 127.0.0.1.5000 > 127.0.0.1.56436: Flags [F.], seq 3975613211, ack 751935907, win 1277, options [nop,nop,TS val 798935022 ecr 3326117646], length 0
2026-09-08 15:11:11.607269 IP 127.0.0.1.56436 > 127.0.0.1.5000: Flags [.], ack 3975613212, win 1277, options [nop,nop,TS val 3326117656 ecr 798935022], length 0
2026-09-08 15:11:21.623193 IP 127.0.0.1.38885 > 127.0.0.1.5000: Flags [S], seq 2754814503, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 4203740687 ecr 0], length 0
2026-09-08 15:11:21.623253 IP 127.0.0.1.5000 > 127.0.0.1.38885: Flags [S.], seq 243721720, ack 2754814504, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 1483227215 ecr 4203740687], length 0
2026-09-08 15:11:21.623307 IP 127.0.0.1.38885 > 127.0.0.1.5000: Flags [.], ack 243721721, win 1277, options [nop,nop,TS val 4203740687 ecr 1483227215], length 0
2026-09-08 15:11:21.625160 IP 127.0.0.1.38885 > 127.0.0.1.5000: Flags [R.], seq 2754814504, ack 243721721, win 0, options [nop,nop,TS val 4203740687 ecr 1483227215], length 0
^C
17 packets captured
< 0 0

```

Figure 16: tcpdump on lo0 covering both parts. Part A (port 56436, 15:11:11) ends with a four-packet FIN exchange; Part B (port 38885, 15:11:21) ends with a single RST about 2 ms after its own SYN. The capture contains three RSTs in total, and only the last one is the SO_LINGER reset under study: at 15:11:11.480553 the server side resets port 23897 because nothing is listening on 5000 yet, and at 15:11:11.590489 the harness's second readiness probe (port 56301) completes a handshake and immediately resets it. Both precede Part A and belong to the harness, not to the experiment.

```

--- Part A: orderly close, client port 56436 -----
15:11:11.593527 56436 > 5000 Flags [S], seq 751935905
15:11:11.593600 5000 > 56436 Flags [S.], seq 3975613210, ack 751935906
15:11:11.593676 56436 > 5000 Flags [.], ack 3975613211
15:11:11.593872 56436 > 5000 Flags [F.], seq 751935906      <- shutdown(SHUT_WR)
15:11:11.593937 5000 > 56436 Flags [.], ack 751935907      <- server ACKs the FIN
15:11:11.607184 5000 > 56436 Flags [F.], seq 3975613211    <- server close()
15:11:11.607269 56436 > 5000 Flags [.], ack 3975613212

--- Part B: abortive close, client port 38885 -----
15:11:21.623193 38885 > 5000 Flags [S], seq 2754814503
15:11:21.623253 5000 > 38885 Flags [S.], seq 243721720, ack 2754814504
15:11:21.623677 38885 > 5000 Flags [.], ack 243721721
15:11:21.625160 38885 > 5000 Flags [R.], seq 2754814504, ack 243721721, win 0

```

```

---
15:11:12
tcp4      0      0 127.0.0.1.56436      127.0.0.1.5000      CLOSED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:13
tcp4      0      0 127.0.0.1.56436      127.0.0.1.5000      CLOSED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:14
tcp4      0      0 127.0.0.1.56436      127.0.0.1.5000      CLOSED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:15
tcp4      0      0 127.0.0.1.56436      127.0.0.1.5000      CLOSED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:16
tcp4      0      0 127.0.0.1.56436      127.0.0.1.5000      CLOSED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:17
tcp4      0      0 127.0.0.1.56436      127.0.0.1.5000      CLOSED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:18
tcp4      0      0 127.0.0.1.56436      127.0.0.1.5000      CLOSED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:19
tcp4      0      0 127.0.0.1.56436      127.0.0.1.5000      CLOSED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:20
tcp4      0      0 127.0.0.1.56436      127.0.0.1.5000      CLOSED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:21

```

Figure 17: After the orderly close (Part A): from 15:11:12 to 15:11:21 the connection remains listed once per second, in state CLOSED, because the client's descriptor is still open.

```

---
15:11:23
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:24
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:25
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:26
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:27
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:28
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:29
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:30
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:31
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:32
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:33
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:34
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
15:11:35

```

Figure 18: After the abortive close (Part B): from 15:11:23 onwards only the LISTEN socket is left. The reset connection was destroyed immediately and never appears in a single snapshot.

```

root@ragnarok:/home/harsh/compiled #
root@ragnarok:/home/harsh/compiled # rm -f /tmp/exp6.ktr
root@ragnarok:/home/harsh/compiled # while :; do pid=$(pgrep -n exchange_server); if [ -n "$pid" ]; then ktrace -f /tmp/exp6.ktr -t cit -p $pid; echo "at
tached to $pid"; break; fi; done
attached to 11815
root@ragnarok:/home/harsh/compiled # kdump -f /tmp/exp6.ktr | grep -n -E 'EV_EOF|errno 54|ECONNRESET|CALL +close'
27: 11815 exchange_server STRU struct kevent[] = { { ident=5, filter=EVFILT_READ, flags=0x8020<EV_CLEAR|EV_EOF>, fflags=0, data=0, udata=0x0 }
44: 11815 exchange_server CALL close(0x5)
64: 11815 exchange_server CALL close(0x3)
66: 11815 exchange_server CALL close(0x4)
root@ragnarok:/home/harsh/compiled # kdump -f /tmp/exp6.ktr | grep -A6 'EV_EOF'
11815 exchange_server STRU struct kevent[] = { { ident=5, filter=EVFILT_READ, flags=0x8020<EV_CLEAR|EV_EOF>, fflags=0, data=0, udata=0x0 }
{ ident=5, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, fflags=0, data=0xbfb8, udata=0x0 } }
11815 exchange_server RET kevent 2
11815 exchange_server CALL read(0x5,0x735f92180c1,0xa0)
11815 exchange_server GIO fd 5 read 0 bytes
""
11815 exchange_server RET read 0
root@ragnarok:/home/harsh/compiled # kdump -f /tmp/exp6.ktr | grep -n -E 'RET +accept'
4: 11815 exchange_server RET accept -1 errno 53 Software caused connection abort
6: 11815 exchange_server RET accept -1 errno 35 Resource temporarily unavailable
13: 11815 exchange_server RET accept 5
24: 11815 exchange_server RET accept -1 errno 35 Resource temporarily unavailable
51: 11815 exchange_server RET accept -1 errno 53 Software caused connection abort
53: 11815 exchange_server RET accept -1 errno 35 Resource temporarily unavailable
root@ragnarok:/home/harsh/compiled #

```

Figure 19: Server-side syscalls. The orderly close surfaces as EVFILT_READ with EV_EOF and read() returning 0 bytes; the abortive one never becomes a connection at all – accept() fails with errno 53, Software caused connection abort. No ECONNRESET appears anywhere in the trace.

```

--- orderly: the server is told politely that the peer is done -----
11815 exchange_server STRU struct kevent[] = { { ident=5, filter=EVFILT_READ,
                                           flags=0x8020<EV_CLEAR|EV_EOF>, fflags=0, data=0, ... } }
11815 exchange_server CALL read(0x5,0x735f92180c1,0xa0)
11815 exchange_server GIO fd 5 read 0 bytes
""
11815 exchange_server RET read 0
11815 exchange_server CALL kevent(0x4, ... EV_DELETE for ident=5 READ and WRITE ... )
11815 exchange_server GIO fd 1 wrote 36 bytes
"Client 127.0.0.1:56436 disconnected"
11815 exchange_server CALL close(0x5)

--- abortive: the reset arrives before the server ever owns a descriptor ---
11815 exchange_server RET accept -1 errno 53 Software caused connection abort
11815 exchange_server RET accept -1 errno 35 Resource temporarily unavailable

```

One field in that trace is worth singling out: the EV_EOF event carries fflags=0. On FreeBSD, kqueue reports a socket error by putting the errno into fflags alongside EV_EOF, so fflags=0 is the kernel stating positively that this was a clean end of stream and not a reset. A connection torn down by an RST after it had been accepted would surface here as fflags=ECONNRESET; none appears anywhere in the trace.

The contrast is complete:

	Orderly (Part A)	Abrupt (Part B)
Trigger	shutdown(SHUT_WR) / close()	SO_LINGER=0 then close()
Packets	FIN → ACK → FIN → ACK (4)	a single RST (1)
Data already sent	delivered before the FIN is reported	discarded
Peer's socket	becomes readable, read() returns 0	destroyed; a later read/write gives ECONNRESET/EPIPE
netstat	visible as CLOSED while the fd lives (Fig. 17)	gone instantly (Fig. 18)
Our server's reaction	EV_EOF, deregister filters, log, close()	accept() returns ECONNABORTED, skipped

Answer. An orderly shutdown is a negotiated, four-packet conversation (FIN/ACK in each direction) that guarantees all previously sent bytes are delivered; the peer's socket becomes readable and read() returns 0, which is how our server learns of it (EVFILT_READ with EV_EOF, followed by close()). An abrupt termination is a **single RST segment**: the connection is

torn down unilaterally with no acknowledgement, any queued data is thrown away, and the entry vanishes from `netstat` immediately instead of passing through the `FIN_WAIT/CLOSED` sequence. In this run the `RST` arrived while the connection was still in the listen queue, so the server saw it as `accept()` failing with `ECONNABORTED` (errno 53); had the connection already been accepted, the next `write()` would have failed with `EPIPE/ECONNRESET` instead. Either way one client's abrupt death is a per-connection error that the event loop absorbs without affecting anyone else.

9 Experiment 7 – Backpressure and the Slow Receiver

How does the server's TCP connection to the slow client behave, and what evidence shows whether this affects communication with other clients?

Approach and tools

The harness connects four clients – a Market-Data Client that keeps draining its socket, a Market-Data Client that *never* reads, and two Traders – and then drives 5000 matching BUY/SELL pairs, so the server must push 5000 `TRADE JNST 1 238` messages (17 bytes each) to *both* subscribers. To give backpressure the best possible chance of appearing, the TCP socket buffers were first shrunk to 8 KiB and auto-tuning was switched off. The run was then instrumented with a set of periodic samplers writing to `/tmp/exp7/`:

```
# 00 -- shrink the socket buffers and switch auto-tuning off, to make the
#       connection reach its buffer limit as early as possible; then record them
$ sysctl net.inet.tcp.recvbuf_auto=0 net.inet.tcp.sendbuf_auto=0
$ sysctl net.inet.tcp.recvspace=8192 net.inet.tcp.sendspace=8192
$ sysctl net.inet.tcp.recvspace net.inet.tcp.sendspace \
    net.inet.tcp.recvbuf_auto net.inet.tcp.sendbuf_auto | tee /tmp/exp7/00_sysctl.txt

# 10 -- watch specifically for zero-window advertisements
$ tcpdump -i lo0 -n -tttt "tcp port 5000 and tcp[14:2] = 0" | tee /tmp/exp7/10_zerowindow.txt

# 20 -- per-connection queue depths, every 2 s
$ while :; do date +%H:%M:%S; netstat -an -p tcp | grep 5000; echo ---; sleep 2; done \
    | tee /tmp/exp7/20_netstat.txt

# 30 -- the run itself
$ python3 experiment.py 7 | tee /tmp/exp7/30_harness.txt

# 40 -- server memory / CPU / what it is blocked on, every 2 s
$ while :; do date +%H:%M:%S; ps -o pid,rss,vsz,%cpu,stat,wchan -p $PID; echo ---; sleep 2; done \
    | tee /tmp/exp7/40_server.txt

# protocol counters before and after, then diff
$ netstat -s -p tcp > /tmp/exp7/01_tcpstats_before.txt # ... run ...
$ netstat -s -p tcp > /tmp/exp7/02_tcpstats_after.txt
$ diff /tmp/exp7/01_tcpstats_before.txt /tmp/exp7/02_tcpstats_after.txt \
    | tee /tmp/exp7/03_tcpstats_diff.txt
```

Every sampler is piped through `tee` rather than redirected, because the screenshots below are the live terminal views of exactly these commands. All of them were run as `root` from a single shell and write into `/tmp/exp7/`, so the files do not end up split across two home directories depending on who ran what.

Observations

```

root@ragnarok:/home/harsh/compiled #
root@ragnarok:/home/harsh/compiled # grep -E 'connected from|Generated' /tmp/exp7/30_harness.txt
Normal Market-Data Client: connected from 127.0.0.1:35877
Slow Market-Data Client: connected from 127.0.0.1:50266
Buyer Trader: connected from 127.0.0.1:65121
Seller Trader: connected from 127.0.0.1:49148
Generated 5000 matching trades.
root@ragnarok:/home/harsh/compiled #

```

Figure 20: The four connections and the generated load: normal Market-Data client on port 35877, slow Market-Data client on port 50266, buyer on 65121, seller on 49148, and **5000 matching trades** generated.

```

root@ragnarok:/home/harsh/compiled # cat /tmp/exp7/00_sysctl.txt
net.inet.tcp.recvspace: 8192
net.inet.tcp.sendspace: 8192
net.inet.tcp.recvbuf_auto: 0
net.inet.tcp.sendbuf_auto: 0
root@ragnarok:/home/harsh/compiled #

```

Figure 21: Socket-buffer parameters recorded at the start of the run.

```

root@ragnarok:/home/harsh/compiled #
root@ragnarok:/home/harsh/compiled # LINE=$(grep -n '85000' /tmp/exp7/20_netstat.txt | head -1 | cut -d: -f1)
root@ragnarok:/home/harsh/compiled # sed -n "${((LINE-7))},${((LINE+7))}p" /tmp/exp7/20_netstat.txt
---
16:49:34
tcp4      0      0 127.0.0.1.5000    127.0.0.1.49148    ESTABLISHED
tcp4      16      0 127.0.0.1.49148    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000    127.0.0.1.65121     ESTABLISHED
tcp4      18      0 127.0.0.1.65121    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000    127.0.0.1.50266     ESTABLISHED
tcp4      85000   0 127.0.0.1.50266    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000    127.0.0.1.35877     ESTABLISHED
tcp4      0      0 127.0.0.1.35877    127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000    *.*                 LISTEN
---
16:49:36
tcp4      0      0 127.0.0.1.5000    127.0.0.1.49148    ESTABLISHED
tcp4      16      0 127.0.0.1.49148    127.0.0.1.5000     ESTABLISHED
root@ragnarok:/home/harsh/compiled #

```

Figure 22: Queue depths at the peak. The slow client's own socket (127.0.0.1.50266) holds Recv-Q = 85000 bytes that its application has never read, while the server's send queues to *all* clients, including the slow one, are essentially empty.

```

root@ragnarok:/home/harsh/compiled #
root@ragnarok:/home/harsh/compiled # head -3 /tmp/exp7/40_server.txt; echo ...; tail -4 /tmp/exp7/40_server.txt
16:46:10
PID    RSS    VSZ    %CPU  STAT  WCHAN
19612  2180  13748  1.1   Ss    kqread
...
16:49:47
PID    RSS    VSZ    %CPU  STAT  WCHAN
19612  2180  13748  0.0   Ss    kqread
...
root@ragnarok:/home/harsh/compiled #

```

Figure 23: The server (PID 19612) at the beginning and the end of the run: RSS unchanged at 2180 KiB, CPU falling back to 0.0%, and WCHAN still kqread.

```

root@ragnarok:/home/harsh/compiled #
root@ragnarok:/home/harsh/compiled # grep -c 'win 0' /tmp/exp7/10_zerowindow.txt
2
root@ragnarok:/home/harsh/compiled # cat /tmp/exp7/10_zerowindow.txt
2026-09-08 16:46:08.760257 IP 127.0.0.1.5000 > 127.0.0.1.27309: Flags [R.], seq 0, ack 539677606, win 0, length 0
2026-09-08 16:46:08.866078 IP 127.0.0.1.53893 > 127.0.0.1.5000: Flags [R.], seq 1531973612, ack 446783982, win 0, options [nop,nop,TS val 1087438920 ecr 2746475297], length 0
root@ragnarok:/home/harsh/compiled #

```

Figure 24: Searching the capture for zero-window advertisements. The only two matches are RST segments (which always carry win 0); no genuine zero-window advertisement was ever sent.

The time series extracted from `20_netstat.txt` is the core result. Writing Q_{recv} for the slow client’s own receive queue and Q_{send} for the server’s send queue towards it:

Time (hh:mm:ss)	16:46:09	16:46:12	16:46:16	16:46:30	16:47:00	16:48:01	16:49:11	16:49:34
Q_{recv} (slow client)	476	1 343	3 060	8 959	21 624	46 750	76 228	85 000
Q_{send} (server side)	17	17	34	17	17	17	34	0

Over all 108 samples of the run:

- The slow client’s **Recv-Q** rises **monotonically** and levels off at **exactly 85 000 bytes** = 5000×17 – every single **TRADE** message was delivered into its socket buffer and none of them was ever read by the application.¹
- The server’s **Send-Q** towards the slow client **never exceeded 34 bytes** (two pending messages) across the entire run, and the same holds for the normal client.
- `netstat -s -p tcp` confirms this from the protocol counters: **0 window probe packets**, **0 window probes** and **0 persist timeouts** – the persist timer never ran, so no receiver ever advertised a closed window.
- The server process was completely unaffected: **RSS** pinned at 2180 KiB for the whole run, **CPU** decaying from 1.1 % to 0.0 %, and **WCHAN** always **kqread** – never **sbwait**, which is what a blocking `write()` would show.
- The other three clients kept working throughout: all 5000 trades completed, the normal subscriber’s queues never exceeded 34 bytes, and the two traders kept getting their **ORDER_ACCEPTED/BOUGHT/SOLD** messages.

Interpretation. Even with the socket buffers deliberately reduced to 8 KiB, the 85 KiB this experiment generates fit entirely inside the slow receiver’s socket buffer, so TCP’s flow control never had to engage on loopback: the receiver kept advertising a non-zero window, the server’s writes all succeeded immediately, and the backlog simply accumulated in the slow client’s own receive buffer. Had the stream been large enough to fill that buffer, the mechanism would have been: receiver advertises **win 0** → the server’s **Send-Q** climbs to the send-buffer limit → `write()` returns **EAGAIN** → the server appends to that connection’s user-space output buffer and waits for **EVFILT_WRITE**. Crucially, none of those steps involve blocking, so the head-of-line blocking stays confined to the one slow connection. A blocking-write server would instead have parked in `write()` with **WCHAN = sbwait** and stopped serving everyone.

Answer. The connection to the slow client stays **ESTABLISHED** and healthy; the unread data piles up on the *receiver’s* side, its **Recv-Q** climbing monotonically to 85 000 bytes – exactly the 5000×17 bytes of market data that the server sent. At this volume the receiver’s socket buffer was never exhausted, so no zero window was ever advertised (**0 window probes**, **0 persist timeouts** in `netstat -s`), and the server’s send queue to that client never exceeded 34 bytes. Communication with the other clients was completely unaffected: all 5000 trades were generated, the normal subscriber’s queues stayed near zero, and the server’s **RSS** (2180 KiB), **CPU** (0 %) and wait channel (**kqread**, i.e. idle in `kevent()`) never changed. Because every write is non-blocking and any residue goes into a per-connection output buffer drained by **EVFILT_WRITE**, a slow reader can only ever penalise itself.

¹The plateau is an order of magnitude above the 8 KiB `net.inet.tcp.recvspace` configured for this run (Figure 21), so the kernel evidently sized this loopback socket’s receive buffer well beyond the nominal value. `recvspace` is only the *initial* size; the hard ceiling is `kern.ipc.maxsockbuf`, which was left at its default and is megabytes wide, so 85 kB is comfortably inside what the socket is allowed to hold. We did not chase the exact sizing rule; what the measurement establishes either way is that the receiver’s buffer was never exhausted, which is what the rest of the argument rests on.

10 Experiment 8 – Unexpected Client Disconnection

When a client disappears unexpectedly, what happens to the TCP connection, and what network-level evidence lets you determine how the server detects the failure?

Approach and tools

The harness starts a *separate process* that acts as a Market-Data Client, subscribes to JNST and then does nothing but sleep, so the server is actively sending it trade updates that it never reads. After 20 trades the harness SIGKILLS that process – the application gets no chance to close anything. A second Market-Data Client and the two traders stay connected so that the two situations can be compared side by side. Packets, socket states and the server's syscalls were all recorded to /tmp/exp8/:

```
# 10 -- full packet capture of the port
$ tcpdump -i lo0 -n -tttt -S tcp port 5000 | tee /tmp/exp8/10_tcpdump.txt

# 20 -- socket table once per second, with timestamps
$ while ;; do date +%H:%M:%S; netstat -an -p tcp | grep 5000; echo ---; sleep 1; done \
| tee /tmp/exp8/20_netstat.txt

# server syscalls -- armed before the run and left spinning, so that ktrace attaches
# within milliseconds of the server appearing and the accept() calls are captured
$ while ;; do pid=$(pgrep -n exchange_server); if [ -n "$pid" ]; then \
ktrace -f /tmp/exp8/server.ktr -t cit -p $pid; echo "attached to $pid"; break; fi; done

# post-processing
$ grep -E 'Flags \[R|Flags \[F' /tmp/exp8/10_tcpdump.txt
$ kdump -f /tmp/exp8/server.ktr | grep -B1 -E 'RET +accept [0-9]'
$ kdump -f /tmp/exp8/server.ktr | sed -n '1620,1650p'
$ awk '/^17:10:02$/,/^17:10:06$/ ' /tmp/exp8/20_netstat.txt
```

Observations

```
root@ragnarok:/home/harsh/compiled # grep 'connected from' /tmp/exp8/30_harness.txt
Surviving Market-Data Client: connected from 127.0.0.1:10123
Buyer Trader: connected from 127.0.0.1:38997
Seller Trader: connected from 127.0.0.1:26991
root@ragnarok:/home/harsh/compiled #
```

Figure 25: The three connections created directly by the harness: surviving Market-Data client on port 10123, buyer on 38997, seller on 26991. The client that will be killed is a separate process and connects on its own.

```
root@ragnarok:/home/harsh/compiled # kdump -f /tmp/exp8/server.ktr | grep -B1 -E 'RET +accept [0-9]'
31814 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:10123 }
31814 exchange_server RET accept 5
--
31814 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:38997 }
31814 exchange_server RET accept 6
--
31814 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:26991 }
31814 exchange_server RET accept 7
--
31814 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:48664 }
31814 exchange_server RET accept 8
root@ragnarok:/home/harsh/compiled #
```

Figure 26: kdump of the server's accept() calls identifies the fourth connection: fd 5 = 10123 (surviving MD), fd 6 = 38997 (buyer), fd 7 = 26991 (seller), fd 8 = 48664 (the client that will disappear).

```

root@ragnarok:/home/harsh/compiled # awk '/^17:10:02$/|^17:10:06$/' /tmp/exp8/20_netstat.txt
17:10:02
tcp4      0      0 127.0.0.1.5000      127.0.0.1.48664      ESTABLISHED
tcp4     136      0 127.0.0.1.48664      127.0.0.1.5000      ESTABLISHED
tcp4      0     16 127.0.0.1.5000      127.0.0.1.26991      ESTABLISHED
tcp4     16      0 127.0.0.1.26991      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.38997      ESTABLISHED
tcp4     18      0 127.0.0.1.38997      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.10123      ESTABLISHED
tcp4      0      0 127.0.0.1.10123      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
17:10:03
tcp4      0      0 127.0.0.1.5000      127.0.0.1.48664      ESTABLISHED
tcp4     255      0 127.0.0.1.48664      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.26991      ESTABLISHED
tcp4     16      0 127.0.0.1.26991      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.38997      ESTABLISHED
tcp4     18      0 127.0.0.1.38997      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.10123      ESTABLISHED
tcp4      0      0 127.0.0.1.10123      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
17:10:04
tcp4      0     34 127.0.0.1.5000      127.0.0.1.26991      ESTABLISHED
tcp4      0      0 127.0.0.1.26991      127.0.0.1.5000      ESTABLISHED
tcp4      0     36 127.0.0.1.5000      127.0.0.1.38997      ESTABLISHED
tcp4      0      0 127.0.0.1.38997      127.0.0.1.5000      ESTABLISHED
tcp4      0     17 127.0.0.1.5000      127.0.0.1.10123      ESTABLISHED
tcp4     17      0 127.0.0.1.10123      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
17:10:05
tcp4      0     34 127.0.0.1.5000      127.0.0.1.26991      ESTABLISHED
tcp4      0      0 127.0.0.1.26991      127.0.0.1.5000      ESTABLISHED
tcp4      0     36 127.0.0.1.5000      127.0.0.1.38997      ESTABLISHED
tcp4      0      0 127.0.0.1.38997      127.0.0.1.5000      ESTABLISHED
tcp4      0     17 127.0.0.1.5000      127.0.0.1.10123      ESTABLISHED

```

Figure 27: netstat one second at a time. At 17:10:02 and 17:10:03 the doomed connection (127.0.0.1.48664) is ESTABLISHED with 136 and then 255 bytes of unread market data in its Recv-Q. At 17:10:04 it is gone completely, while the other three connections carry on exchanging data.

```

root@ragnarok:/home/harsh/compiled # grep -E 'Flags \[R\]Flags \[F\]' /tmp/exp8/10_tcpdump.txt
2026-09-08 17:09:59.757302 IP 127.0.0.1.5000 > 127.0.0.1.27571: Flags [R.], seq 0, ack 4090159394, win 0, length 0
2026-09-08 17:09:59.862060 IP 127.0.0.1.59148 > 127.0.0.1.5000: Flags [R.], seq 1528691534, ack 2433209448, win 0, options [nop,nop,TS val 1409265845 ecr 158687807], length 0
2026-09-08 17:10:03.772273 IP 127.0.0.1.48664 > 127.0.0.1.5000: Flags [F.], seq 584026206, ack 2245618379, win 1277, options [nop,nop,TS val 3883213896 ecr 700284551], length 0
2026-09-08 17:10:03.776622 IP 127.0.0.1.5000 > 127.0.0.1.48664: Flags [F.], seq 2245618379, ack 584026207, win 1277, options [nop,nop,TS val 700284692 ecr 3883213896], length 0
2026-09-08 17:10:23.524663 IP 127.0.0.1.10123 > 127.0.0.1.5000: Flags [F.], seq 676359951, ack 3413299175, win 1277, options [nop,nop,TS val 823780914 ecr 152491013], length 0
2026-09-08 17:10:23.524712 IP 127.0.0.1.38997 > 127.0.0.1.5000: Flags [F.], seq 2755748733, ack 3346130460, win 1277, options [nop,nop,TS val 827532680 ecr 3480140277], length 0
2026-09-08 17:10:23.524739 IP 127.0.0.1.26991 > 127.0.0.1.5000: Flags [F.], seq 3239146630, ack 3501322117, win 1277, options [nop,nop,TS val 2552839797 ecr 1252052305], length 0
2026-09-08 17:10:23.525699 IP 127.0.0.1.5000 > 127.0.0.1.10123: Flags [F.], seq 3413299175, ack 676359952, win 1277, options [nop,nop,TS val 152506135 ecr 823780914], length 0
2026-09-08 17:10:23.526754 IP 127.0.0.1.5000 > 127.0.0.1.38997: Flags [F.], seq 3346130460, ack 2755748734, win 1277, options [nop,nop,TS val 3480155361 ecr 827532680], length 0
2026-09-08 17:10:23.527424 IP 127.0.0.1.5000 > 127.0.0.1.26991: Flags [F.], seq 3501322117, ack 3239146631, win 1277, options [nop,nop,TS val 1252067389 ecr 2552839797], length 0
root@ragnarok:/home/harsh/compiled #

```

Figure 28: Every FIN and RST on the port. At 17:10:03.772273 the killed client's kernel sends a FIN, the server ACKs it and replies with its own FIN 4 ms later. The other three connections only close at 17:10:23, when the experiment ends.

```

root@ragnarok:/home/harsh/compiled # kdump -f /tmp/exp8/server.ktr | sed -n '1620,1650p'
31814 exchange_server CALL kevent(0x4,0,0,0x82085e0c0,0x40,0)
31814 exchange_server STRU struct kevent[] = { }
31814 exchange_server STRU struct kevent[] = { { ident=6, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, fflags=0, data=0xbf88, udata=0x0 }
{ ident=7, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, fflags=0, data=0xbf88, udata=0x0 } }
31814 exchange_server RET kevent 2
31814 exchange_server CALL kevent(0x4,0,0,0x82085e0c0,0x40,0)
31814 exchange_server STRU struct kevent[] = { }
31814 exchange_server STRU struct kevent[] = { { ident=8, filter=EVFILT_READ, flags=0x8020<EV_CLEAR|EV_EOF>, fflags=0, data=0, udata=0x0 } }
31814 exchange_server RET kevent 1
31814 exchange_server CALL read(0x8,0x3172b7018541,0xa0)
31814 exchange_server GIO fd 8 read 0 bytes
""
31814 exchange_server RET read 0
31814 exchange_server CALL kevent(0x4,0x82085e000,0x2,0,0,0)
31814 exchange_server STRU struct kevent[] = { { ident=8, filter=EVFILT_READ, flags=0x2<EV_DELETE>, fflags=0, data=0, udata=0x0 }
{ ident=8, filter=EVFILT_WRITE, flags=0x2<EV_DELETE>, fflags=0, data=0, udata=0x0 } }
31814 exchange_server STRU struct kevent[] = { }
31814 exchange_server RET kevent 0
31814 exchange_server CALL write(0x1,0x3172b7009000,0x24)
31814 exchange_server GIO fd 1 wrote 36 bytes
"Client 127.0.0.1:48664 disconnected"
""
31814 exchange_server RET write 36/0x24
31814 exchange_server CALL close(0x8)
31814 exchange_server RET close 0
31814 exchange_server CALL kevent(0x4,0,0,0x82085e0c0,0x40,0)
31814 exchange_server STRU struct kevent[] = { }
31814 exchange_server STRU struct kevent[] = { { ident=6, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 }
{ ident=7, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0x10, udata=0x0 } }
31814 exchange_server RET kevent 2
31814 exchange_server CALL read(0x6,0x3172b7018241,0xa0)
root@ragnarok:/home/harsh/compiled #

```

Figure 29: The server's view of the death: `kevent()` reports `ident=8` with `EV_CLEAR|EV_EOF`, `read(8)` returns **0 bytes**, the filters are deleted, the disconnect is logged and `close(8)` is called. The very next `kevent()` already returns readiness for `ident=6` and `ident=7` – the other clients were never interrupted.

The complete sequence around the kill:

```

--- packets -----
17:10:03.632762 5000 > 48664 Flags [P.], length 17 "TRADE JNST 1 238"
17:10:03.726335 48664 > 5000 Flags [.], ack ... (kernel ACKs; app never reads)
17:10:03.772273 48664 > 5000 Flags [F.], seq 584026206 <- process is SIGKILLED here
17:10:03.772319 5000 > 48664 Flags [.], ack 584026207
17:10:03.776622 5000 > 48664 Flags [F.], seq 2245618379
17:10:03.776710 48664 > 5000 Flags [.], ack 2245618380

--- socket table -----
17:10:03 tcp4 255 0 127.0.0.1.48664 127.0.0.1.5000 ESTABLISHED
17:10:04 (no row for 48664 at all; 26991, 38997 and 10123 still ESTABLISHED
with Send-Q 34, 36 and 17 bytes -- traffic continues normally)

--- server syscalls -----
31814 exchange_server STRU struct kevent[] = { { ident=8, filter=EVFILT_READ,
flags=0x8020<EV_CLEAR|EV_EOF>, fflags=0, data=0, ... } }
31814 exchange_server RET kevent 1
31814 exchange_server CALL read(0x8,0x3172b7018541,0xa0)
31814 exchange_server GIO fd 8 read 0 bytes
""
31814 exchange_server RET read 0
31814 exchange_server CALL kevent(...) EV_DELETE for ident=8 READ and WRITE
31814 exchange_server GIO fd 1 wrote 36 bytes
"Client 127.0.0.1:48664 disconnected"
31814 exchange_server CALL close(0x8)
31814 exchange_server CALL kevent(0x4,0,0,0x82085e0c0,0x40,0)
31814 exchange_server STRU struct kevent[] = { { ident=6, filter=EVFILT_READ, ..., data=0xf }
{ ident=7, filter=EVFILT_READ, ..., data=0x10 } }

```

Three things deserve emphasis.

- **The connection was closed by the kernel, not the application.** The client process was SIGKILLED, so it executed no shutdown code at all. FreeBSD still released its file descriptors on process exit, and the TCP stack performed a normal active close. On the wire an abruptly killed *process* therefore looks like an orderly close: a FIN, not an RST.
- **Unread data is visible right up to the moment of death.** Recv-Q on the doomed

socket grows 17 → 136 → 255 bytes as trades arrive that the sleeping process never reads, and one second later the socket is gone entirely – no `TIME_WAIT` row is left behind, for the same `net.inet.tcp.nolocaltimewait` reason established in Experiment 2 (page 7): the peer address is local, so the endpoint that performed the active close skips `TIME_WAIT` altogether.

- **Detection is immediate and local.** The server did not need a timeout, a keep-alive or a failed write: the `FIN` made the descriptor readable, `read()` returned 0, and the connection slot was released. The `EV_EOF` event again carries `fflags=0`, and searching the whole trace for `errno 32` (`EPIPE`) or `errno 54` (`ECONNRESET`) returns nothing – the server never even attempted a write that could fail. The next `kevent()` call in the same trace is already servicing fds 6 and 7, and the surviving subscriber received all 50 of the post-kill trades.

The harder failure mode. This experiment kills a process, and the OS is still there to send the `FIN`. If instead the *machine* died or the network were cut, no `FIN` and no `RST` would ever arrive; the server's socket would stay `ESTABLISHED` indefinitely. It would then only find out by writing – retransmissions time out and the connection is dropped with `ETIMEDOUT`, or the peer reboots and answers with an `RST`, giving `ECONNRESET/EPIPE` – or by enabling `SO_KEEPALIVE` and letting `net.inet.tcp.keepidle/keepintvl/keepcnt` probe the peer.

Answer. The client process vanished without running any shutdown code, but the FreeBSD kernel closed its descriptors on exit, so the TCP connection was terminated with a normal **FIN exchange** at 17:10:03.772, not a reset – the server `ACKed` and sent its own `FIN` 4 ms later, and the socket disappeared from `netstat` within one second (having held 255 bytes of undelivered market data at the moment of death). The server detected the failure through that `FIN`: the descriptor became readable with `EVFILT_READ | EV_EOF` and `read()` returned 0 bytes, whereupon it deregistered the filters, logged `Client 127.0.0.1:48664 disconnected` and closed fd 8. The other three connections were not disturbed at all – the very next `kevent()` return in the same trace services fds 6 and 7.

11 Summary of Tools Used

Tool	Experiments	What it contributed
<code>sockstat -4 -l / -c</code>	1, 4	Maps every socket to its owning process and file descriptor; separates listening from connected sockets.
<code>netstat -an -p tcp</code>	1, 2, 4, 5, 6, 7, 8	TCP state column, and the <code>Recv-Q/Send-Q</code> depths used as evidence that a partial message had already been consumed by the server (Exp. 4), of readiness (Exp. 5) and of backpressure (Exp. 7, 8).
<code>netstat -s -p tcp</code>	7	Protocol counters: 0 window probes and 0 persist timeouts proved no zero-window event occurred.
<code>tcpdump -i lo0</code>	2, 3, 6, 7, 8	The packet-level ground truth: handshake, per-segment framing (<code>-A</code>), FIN vs. RST, zero-window search.
<code>ktrace + kdump</code>	5, 6, 8	The server's own syscalls: <code>kevent()</code> return values naming the ready descriptors, <code>read()</code> returning 0 at EOF, <code>accept()</code> <code>errno</code> values.
<code>procstat -k</code>	4	Kernel stack of the server thread – proved it sleeps in <code>kqueue_scan</code> , not in <code>recv()</code> .
<code>procstat -f</code>	5	Descriptor table, used to map <code>kevent</code> ids back to real client sockets.
<code>ps -H -o ...,wchan</code>	4, 7	Thread count, <code>WCHAN</code> (<code>kqread</code>), RSS and CPU of the server under load.
<code>sysctl</code>	7	Socket-buffer parameters that gate backpressure.

A Bonus – Connection Scalability and I/O Design

This appendix reports the results of Section 6.9 of the handout: driving the Exchange Server up to $N = 70\,000$ simultaneous idle TCP connections, measuring the per- N resource footprint, and analysing the bottlenecks reached. All measurements were taken on the same FreeBSD 14.4-RELEASE VM used for Experiments 1–8. A companion document, `bonus/README.md` in the submission, lists every OS-level tunable and command verbatim; only the parts needed to read the tables and figures are repeated here.

A.1 Setup and tooling

Two small helper programs, both in `bonus/`, were written for this section. The Exchange Server, Trader Client and Market-Data Client themselves are otherwise unchanged from Experiments 1–8 (with the single exception noted in §A.2).

bonus/idle_clients.py – client generator. Opens N TCP connections to the Exchange Server and holds them ESTABLISHED without ever sending or receiving application data. It raises `RLIMIT_NOFILE` to $N + 128$ before starting, reports progress and connect rate every `-batch` connections, and records the index and `errno` of every `connect()` failure so the count at which the server first fails is unambiguous. Three additive optional flags (`--src-ips`, `--src-cidr`, `--src-count`) enable source-IP rotation and are described in §A.3; without them the script behaves exactly like a default client (kernel-picked 127.0.0.1 + ephemeral port).

```
$ python3 bonus/idle_clients.py 127.0.0.1 5000 50000 --batch 2000
$ python3 bonus/idle_clients.py 127.0.0.1 5000 70000 --batch 2000 \
  --src-cidr 127.0.0.0/8 --src-count 4
```

bonus/measure.sh – one snapshot per row of the table. Runs while `idle_clients.py` is holding its connections open and captures every column of the required Section 6.9 table in a single output:

```
$ ./bonus/measure.sh <server-pid> 5000 | tee run_<N>.txt

established conns    netstat -an -p tcp | awk ... | wc -l  (+ sockstat cross-check)
server FDs          procstat -f <pid> | tail -n +2 | wc -l
memory / CPU        ps -o pid,rss,vsz,%cpu,%mem,nlwp,command -p <pid>
system-wide FDs     sysctl kern.openfiles kern.maxfiles kern.maxfilesperproc
socket buffers       netstat -m
                    sysctl kern.ipc.maxsockbuf net.inet.tcp.sendspace \
                    net.inet.tcp.recvspace kern.ipc.somaxconn
I/O design check    procstat -k <pid>
```

OS tuning applied. FreeBSD’s defaults (~64 k sockets, ~29 k FDs per proc, `SOMAXCONN=128`) cannot support 70 k connections. The tunables raised before the runs are summarised in Table 1; full rationale is in `bonus/README.md`.

Where	Setting	Reboot?
/boot/loader.conf	kern.ipc.maxsockets=400000	Yes
/boot/loader.conf	kern.ipc.nmbclusters=500000	Yes
/etc/sysctl.conf	kern.ipc.somaxconn=65535	No
/etc/sysctl.conf	kern.maxfiles=300000	No
/etc/sysctl.conf	kern.maxfilesperproc=250000	No
/etc/sysctl.conf	net.inet.ip.portrange.first=10000	No
/etc/sysctl.conf	net.inet.ip.portrange.last=65535	No
Per-shell (server + gen.)	ulimit -n 250000	No
lo0 aliases	ifconfig lo0 alias 127.0.0.{2,3,4}/32	No

Table 1: OS-level tunables applied for the bonus experiment. Applied once as root on the FreeBSD 14 VM; see `bonus/README.md` §3–§5 for the exact commands and persistence recipes.

A.2 Changes made to the Exchange Server

Exactly one line of the Exchange Server was changed for the bonus, and only after the 40 k run made the need explicit. The original code called

```
listen(server_fd, SOMAXCONN); /* SOMAXCONN is the C-header macro = 128 on FreeBSD 14 */
```

On FreeBSD 14 the C header defines `SOMAXCONN` as **128**, *independent* of the runtime `sysctl kern.ipc.somaxconn` that Table 1 raises to 65 535. Under the sustained ~55 000/s SYN flood produced by `idle_clients.py`, the 128-entry syncache overflowed during the 40 k run and **199 out of 40 000** SYNs were RST’d by the kernel in a single burst starting at attempt #30 994 (see Fig. 34). The fix is one character-count change:

```
if (listen(server_fd, 65535) == -1) { ... } /* was: SOMAXCONN */
```

The kernel silently clamps the argument to `kern.ipc.somaxconn`, so 65535 is safe on any FreeBSD. **Scope of the change:** exactly one syscall argument, sizing the syncache; the accept path, `kqueue` loop, message framing and order-matching code are byte-for-byte identical. The 10 k / 20 k / 30 k / 40 k rows below were collected with the original `SOMAXCONN` build; every row from 50 k onward uses the new `listen(65535)` build. The direct A/B is 40 k → 199 lost vs. 50 k → 0 lost – the change eliminated the losses even at higher load. No other server-side change was made.

A.3 Client-side source-IP rotation (for $N > 55\,000$)

A single `idle_clients.py` process connecting from `127.0.0.1` to `127.0.0.1:5000` cannot exceed about **55 000 connections** in one run. That is a TCP 4-tuple constraint, not a server limit: with $(src_ip, src_port, dst_ip, dst_port)$ required to be unique and three of the four values fixed, only `src_port` can vary, and only $\sim 55\,000$ ephemeral ports exist even after widening `net.inet.ip.portrange`.

To reach 60 k and 70 k in a single process, `idle_clients.py` was extended with three *optional* flags – `--src-ips`, `--src-cidr`, `--src-count` – that round-robin `s.bind((src_ip, 0))` before `connect()`. Each distinct `src_ip` has its own ephemeral-port space, so K source IPs raise the single-run ceiling to $\sim K \times 55\,000$. Default behaviour (all earlier runs, 10 k–50 k) is unchanged.

FreeBSD prerequisite. Unlike Linux, FreeBSD does not accept `bind((127.x.x.x, 0))` unless the address is explicitly aliased on `lo0` (`EADDRNOTAVAIL` otherwise). Once, as root:

```
ifconfig lo0 alias 127.0.0.2/32
ifconfig lo0 alias 127.0.0.3/32
ifconfig lo0 alias 127.0.0.4/32
```

The 60 k and 70 k rows below therefore invoke `--src-cidr 127.0.0.0/8 --src-count 4`, drawing from $\{127.0.0.1, 127.0.0.2, 127.0.0.3, 127.0.0.4\}$. The server side sees no difference from the earlier rows – it is still just `accept()`ing loopback connections.

A.4 Full measurement table

All rows below come from `measure.sh` run against a live server that `idle_clients.py` is holding open at exactly N established connections. Column definitions match the Section 6.9 hand-out: server RSS and %CPU from `ps`, server FDs from `procstat -f`, system-wide FDs from `sysctl kern.openfiles`, socket-buffer usage from `netstat -m` (mbuf clusters in use / cap), and the established count cross-checked between `sockstat` (minus one for the listening socket) and `idle_clients.py`’s own counter.

N	RSS	%CPU	Server FDs	Sys FDs (used/max)	mbuf clusters (used/max)	Net alloc	Established
10 000	6 140 KB	0.0 %	10 009	20 094 / 300 000	2 049 / 500 000	5 644 KB	10 000 / 10 000
20 000	9 996 KB	0.0 %	20 008	40 092 / 300 000	2 051 / 500 000	5 648 KB	19 999 / 20 000 [†]
30 000	13 968 KB	0.0 %	30 009	60 176 / 300 000	2 049 / 500 000	5 644 KB	30 000 / 30 000
40 000	17 676 KB	0.0 %	39 806	79 774 / 300 000	2 050 / 500 000	5 646 KB	39 797 / 40 000 [†]
50 000	21 888 KB	0.0 %	50 009	100 197 / 300 000	2 049 / 500 000	5 644 KB	50 000 / 50 000 [‡]
60 000	25 696 KB	0.2 %	60 009	120 198 / 300 000	2 050 / 500 000	5 646 KB	60 000 / 60 000 ^{‡§}
70 000	29 512 KB	0.1 %	70 009	140 198 / 300 000	2 049 / 500 000	5 644 KB	70 000 / 70 000^{‡§}

Table 2: Resource footprint of the Exchange Server holding N idle TCP connections, $N = 10\,000 \dots 70\,000$.

[†] Original `listen(SOMAXCONN)` build; transient listen-backlog RSTs during the SYN flood (1 at 20 k, 199 at 40 k) – see §A.2. [‡] `listen(65535)` build. [§] Source-IP rotation across `127.0.0.{1..4}` (see §A.3). Column definitions in §A.4.

Growth per 10 k connections. Reading across Table 2: server RSS grows by ≈ 3.9 MB per 10 000 connections, i.e. **~ 390 B/conn** of user-space memory (essentially the size of `struct ClientConnection` + one `conns[]` slot). Server FDs grow by exactly one per connection. System-wide FDs grow by two per connection because loopback holds both the client end and the accepted server end. *mbuf clusters do not grow at all*: they stay at $\sim 2\,050$ across every row – idle sockets *reserve* socket-buffer space but do not *allocate* clusters until data flows. %CPU is 0 at steady state at every N ; the server thread spends its entire life parked inside a single `kevent()` call (`procstat -k` confirms the kernel stack `mi_switch` $\rightarrow$ `sleepq_wait_sig` $\rightarrow$ `kqueue_scan` $\rightarrow$ `sys_kevent` at every scale).

A.5 Screenshots

Three representative rows are shown as raw `measure.sh` output: the smallest ($N = 10\,000$), the row that exposed the listen-backlog bug ($N = 40\,000$), and the target ($N = 70\,000$). The remaining rows are in `bonus/` as `run_20k.txt` through `run_60k.txt`.

```

root@ragnarok:/home/harsh/compiled/bonus # python3 idle_clients.py 127.0.0.1 5000 10000 --batch 1000
[+] target: 10000 idle TCP connections -> 127.0.0.1:5000
[+] client PID = 942 (use for 'procstat -f 942' etc.)
[+] RLIMIT_NOFILE now soft=250000 hard=250000
[i] FreeBSD tuning hints if you hit failures at large N:
    sysctl kern.maxfiles kern.maxfilesperproc kern.ipc.somaxconn
    sysctl net.inet.ip.portrange.first net.inet.ip.portrange.last
    sysctl kern.ipc.maxsockbuf net.inet.tcp.sendspace net.inet.tcp.recvspace
    (raise them via /etc/sysctl.conf or 'sudo sysctl KEY=VAL' before running)
    established= 1000 failed= 0 elapsed= 0.0s rate= 29845/s
    established= 2000 failed= 0 elapsed= 0.1s rate= 35584/s
    established= 3000 failed= 0 elapsed= 0.1s rate= 38319/s
    established= 4000 failed= 0 elapsed= 0.2s rate= 20172/s
    established= 5000 failed= 0 elapsed= 0.2s rate= 23331/s
    established= 6000 failed= 0 elapsed= 0.2s rate= 26964/s
    established= 7000 failed= 0 elapsed= 0.2s rate= 28699/s
    established= 8000 failed= 0 elapsed= 0.3s rate= 30581/s
    established= 9000 failed= 0 elapsed= 0.3s rate= 32784/s
    established= 10000 failed= 0 elapsed= 0.3s rate= 34183/s

[+] connect phase done: established=10000 failed=0 elapsed=0.3s
[+] client PID = 942
[+] holding 10000 sockets open. Ctrl-C to close all and exit.

While this holds, in another terminal you can run e.g.:
    sockstat -4 -c | wc -l
    netstat -an -p tcp | awk '$6=="ESTABLISHED"' | wc -l
    procstat -f <server-pid> | wc -l
    ps -o pid,rss,vsz,%cpu -p <server-pid>
    netstat -m
    sysctl kern.openfiles kern.maxfiles
[.] holding: sockets_alive=10000 pid=942 uptime=30s

```

Figure 30: $N = 10\,000$: `measure.sh` output part 1 – `ps`, `procstat -f`, `sysctl kern.openfiles`. RSS=6 140 KB, %CPU=0.0, server FDs = 10 009, system-wide FDs = 20 094 / 300 000.

```

root@ragnarok:/home/harsh/compiled/bonus # SPID=$(pgrep -f exchange_server | head -1)
root@ragnarok:/home/harsh/compiled/bonus # ./measure.sh $SPID 5000 | tee run_10k.txt

==== date / uname ====
Wed Sep  9 19:26:59 IST 2026
FreeBSD ragnarok 14.4-RELEASE FreeBSD 14.4-RELEASE releng/14.4-n273675-a456f852d145 GENERIC amd64

==== established connections to server port 5000 ====
ESTABLISHED_to_5000 = 0

==== server open FDs (procstat -f | wc -l) ====
server_open_fds = 10009

==== server memory / cpu (ps) ====
PID  RSS   VSZ %CPU %MEM NLWP  COMMAND
941 6140 16308  0.0  0.3   1  ./exchange_server 127.0.0.1 5000

==== system-wide open files (sysctl kern.openfiles / kern.maxfiles) ====
kern.openfiles: 20094
kern.maxfiles: 300000
kern.maxfilesperproc: 250000

```

Figure 31: $N = 10\,000$: `netstat -m` and socket-buffer sysctls – 2049 mbuf clusters in use / 500 000 cap; 5 644 KB allocated to network. All 10 000 idle connections consume < 0.5 % of the mbuf cluster pool.

```

==== mbuf / socket-buffer usage (netstat -m) ====
6186/1434/7620 mbufs in use (current/cache/total)
2849/745/2794/500000 mbuf clusters in use (current/cache/total/max)
1/507 mbuf+clusters out of packet secondary zone in use (current/cache)
0/762/762/62503 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/18519 9k jumbo clusters in use (current/cache/total/max)
0/0/0/10417 16k jumbo clusters in use (current/cache/total/max)
5644K/4896K/10541K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 sendfile syscalls
0 sendfile syscalls completed without I/O request
0 requests for I/O initiated by sendfile
0 pages read by sendfile as part of a request
0 pages were valid at time of a sendfile request
0 pages were valid and substituted to bogus page
0 pages were requested for read ahead by applications
0 pages were read ahead by sendfile
0 times sendfile encountered an already busy page

==== socket-buffer sysctls ====
kern.ipc.maxsockbuf: 2097152
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
kern.ipc.somaxconn: 65535

==== server kernel stack (where is it blocked?) ====
PID   TID  COMM          TDNAME          KSTACK
941 100258 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_
kevent_generic sys_kevent amd64_syscall fast_syscall_common

==== sockstat summary for server (count) ====
server_sockets_seen_by_sockstat = 10001
root@ragnarok:/home/harsh/compiled/bonus #

```

Figure 32: $N = 10\,000$: `procstat -k` of the server thread and `sockstat` count. The single thread is parked at `sys_kevent` (kqueue idle wait); `sockstat` sees 10001 sockets (10000 clients + 1 listener), confirming the connect count.

```

root@ragnarok:/home/harsh/compiled/bonus # python3 idle_clients.py 127.0.0.1 5000 40000 --batch 1000
[+] target: 40000 idle TCP connections -> 127.0.0.1:5000
[+] client PID = 1743 (use for `procstat -f 1743` etc.)
[+] RLIMIT_NOFILE now soft=250000 hard=250000
[i] FreeBSD tuning hints if you hit failures at large N:
sysctl kern.maxfiles kern.maxfilesperproc kern.ipc.somaxconn
sysctl net.inet.ip.portrange.first net.inet.ip.portrange.last
sysctl kern.ipc.maxsockbuf net.inet.tcp.sendspace net.inet.tcp.recvspace
(raise them via /etc/sysctl.conf or `sudo sysctl KEY=VAL` before running)
established= 1000 failed= 0 elapsed= 0.0s rate= 56408/s
established= 2000 failed= 0 elapsed= 0.0s rate= 70600/s
established= 3000 failed= 0 elapsed= 0.0s rate= 69585/s
established= 4000 failed= 0 elapsed= 0.1s rate= 68877/s
established= 5000 failed= 0 elapsed= 0.1s rate= 74061/s
established= 6000 failed= 0 elapsed= 0.1s rate= 56269/s
established= 7000 failed= 0 elapsed= 0.1s rate= 57706/s
established= 8000 failed= 0 elapsed= 0.1s rate= 59527/s
established= 9000 failed= 0 elapsed= 0.2s rate= 57133/s
established= 10000 failed= 0 elapsed= 0.2s rate= 59270/s
established= 11000 failed= 0 elapsed= 0.2s rate= 59654/s
established= 12000 failed= 0 elapsed= 0.2s rate= 59005/s
established= 13000 failed= 0 elapsed= 0.2s rate= 59829/s
established= 14000 failed= 0 elapsed= 0.2s rate= 60456/s
established= 15000 failed= 0 elapsed= 0.3s rate= 54245/s
established= 16000 failed= 0 elapsed= 0.3s rate= 48347/s
established= 17000 failed= 0 elapsed= 0.3s rate= 48999/s
established= 18000 failed= 0 elapsed= 0.4s rate= 49408/s
established= 19000 failed= 0 elapsed= 0.4s rate= 49735/s
established= 20000 failed= 0 elapsed= 0.4s rate= 49798/s
established= 21000 failed= 0 elapsed= 0.4s rate= 50688/s
established= 22000 failed= 0 elapsed= 0.4s rate= 51873/s
established= 23000 failed= 0 elapsed= 0.4s rate= 52265/s
established= 24000 failed= 0 elapsed= 0.5s rate= 52914/s
established= 25000 failed= 0 elapsed= 0.5s rate= 52703/s
established= 26000 failed= 0 elapsed= 0.5s rate= 52336/s
established= 27000 failed= 0 elapsed= 0.5s rate= 53299/s
established= 28000 failed= 0 elapsed= 0.5s rate= 53463/s

```

Figure 33: $N = 40\,000$ (original `listen(SOMAXCONN)` build): `ps + procstat -f`. RSS=17 676 KB, server FDs = 39 806, system-wide FDs = 79 774 / 300 000.

```

established= 18000 failed= 0 elapsed= 0.4s rate= 49408/s
established= 19000 failed= 0 elapsed= 0.4s rate= 49735/s
established= 20000 failed= 0 elapsed= 0.4s rate= 49798/s
established= 21000 failed= 0 elapsed= 0.4s rate= 50688/s
established= 22000 failed= 0 elapsed= 0.4s rate= 51873/s
established= 23000 failed= 0 elapsed= 0.4s rate= 52265/s
established= 24000 failed= 0 elapsed= 0.5s rate= 52914/s
established= 25000 failed= 0 elapsed= 0.5s rate= 52703/s
established= 26000 failed= 0 elapsed= 0.5s rate= 52336/s
established= 27000 failed= 0 elapsed= 0.5s rate= 53299/s
established= 28000 failed= 0 elapsed= 0.5s rate= 53463/s
established= 29000 failed= 0 elapsed= 0.5s rate= 54116/s
established= 30000 failed= 0 elapsed= 0.5s rate= 54611/s
[!] connect #30994 failed: 54 Connection reset by peer
[!] connect #30995 failed: 54 Connection reset by peer
[!] connect #30996 failed: 54 Connection reset by peer
[!] connect #30997 failed: 54 Connection reset by peer
[!] connect #30998 failed: 54 Connection reset by peer
established= 30994 failed= 6 elapsed= 0.6s rate= 55267/s
established= 31801 failed= 199 elapsed= 0.6s rate= 55246/s
established= 32801 failed= 199 elapsed= 0.6s rate= 54801/s
established= 33801 failed= 199 elapsed= 0.6s rate= 54668/s
established= 34801 failed= 199 elapsed= 0.7s rate= 53424/s
established= 35801 failed= 199 elapsed= 0.7s rate= 52361/s
established= 36801 failed= 199 elapsed= 0.7s rate= 51445/s
established= 37801 failed= 199 elapsed= 0.7s rate= 50803/s
established= 38801 failed= 199 elapsed= 0.8s rate= 50699/s
established= 39801 failed= 199 elapsed= 0.8s rate= 50580/s

[+] connect phase done: established=39801 failed=199 elapsed=0.8s
[+] first failure was at attempt #30994
[+] client PID = 1743
[+] holding 39801 sockets open. Ctrl-C to close all and exit.

```

Figure 34: $N = 40\,000$: `idle_clients.py` output during the run – 199 `ECONNRESET` failures in a single burst starting at attempt #30994. Cause: syncache overflow against the C-header `SOMAXCONN=128`, unrelated to the tuned `sysctl` (65535). Fixed by the one-line change in §A.2.

```

root@ragnarok:/home/harsh/compiled/bonus # ./measure.sh $SPID 5000 | tee run_40k.txt

==== date / uname ====
Wed Sep  9 19:54:58 IST 2026
FreeBSD ragnarok 14.4-RELEASE FreeBSD 14.4-RELEASE releng/14.4-n273675-a456f852d145 GENERIC amd64

==== established connections to server port 5000 ====
ESTABLISHED_to_5000 = 0

==== server open FDs (procstat -f | wc -l) ====
server_open_fds = 39806

==== server memory / cpu (ps) ====
  PID  RSS  VSZ %CPU %MEM NLWP COMMAND
1734 17676 32180 0.0 0.9   1 ./exchange_server 127.0.0.1 5000

==== system-wide open files (sysctl kern.openfiles / kern.maxfiles) ====
kern.openfiles: 79774
kern.maxfiles: 300000
kern.maxfilesperproc: 250000

```

Figure 35: $N = 40\,000$: `netstat -m` – 2050 mbuf clusters in use, essentially unchanged from the 10k row. Idle connections still do not consume clusters even after $4\times$ scale-up.

```

==== mbuf / socket-buffer usage (netstat -m) ====
6186/1944/8130 mbufs in use (current/cache/total)
2050/1252/3302/500000 mbuf clusters in use (current/cache/total/max)
2/760 mbuf+clusters out of packet secondary zone in use (current/cache)
0/762/762/62503 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/18519 9k jumbo clusters in use (current/cache/total/max)
0/0/0/10417 16k jumbo clusters in use (current/cache/total/max)
5646K/6038K/11684K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 sendfile syscalls
0 sendfile syscalls completed without I/O request
0 requests for I/O initiated by sendfile
0 pages read by sendfile as part of a request
0 pages were valid at time of a sendfile request
0 pages were valid and substituted to bogus page
0 pages were requested for read ahead by applications
0 pages were read ahead by sendfile
0 times sendfile encountered an already busy page

==== socket-buffer sysctls ====
kern.ipc.maxsockbuf: 2097152
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
kern.ipc.somaxconn: 65535

==== server kernel stack (where is it blocked?) ====
PID    TID COMM          TDNAME          KSTACK
1734 100316 exchange_server -              mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_
kevent_generic sys_kevent amd64_syscall fast_syscall_common

==== sockstat summary for server (count) ====
server_sockets_seen_by_sockstat = 39798
root@ragnarok:/home/harsh/compiled/bonus # clear

```

Figure 36: $N = 40\,000$: `procstat -k` + `sockstat`. Kernel stack is still parked at `sys_kevent` despite 39797 established connections; %CPU still 0.0.

```

root@ragnarok:/home/harsh/compiled/bonus #
root@ragnarok:/home/harsh/compiled/bonus # python3 idle_clients.py 127.0.0.1 5000 70000 --batch 2000 --src-cidr 127.0.0.0/8 --src-count 4
[+] target: 70000 idle TCP connections -> 127.0.0.1:5000
[+] client PID = 4127 (use for 'procstat -f 4127' etc.)
[+] rotating source IPs over pool of 4: 127.0.0.1, 127.0.0.2, 127.0.0.3, 127.0.0.4
[+] theoretical single-run ceiling raised to ~220,000 connections
[+] RLIMIT_NOFILE now soft=250000 hard=250000
[i] FreeBSD tuning hints if you hit failures at large N:
sysctl kern.maxfiles kern.maxfilesperproc kern.ipc.somaxconn
sysctl net.inet.ip.portrange.first net.inet.ip.portrange.last
sysctl kern.ipc.maxsockbuf net.inet.tcp.sendspace net.inet.tcp.recvspace
(raise them via /etc/sysctl.conf or 'sudo sysctl KEY=VAL' before running)
established= 2000 failed= 0 elapsed= 0.1s rate= 19864/s
established= 4000 failed= 0 elapsed= 0.1s rate= 27084/s
established= 6000 failed= 0 elapsed= 0.2s rate= 33101/s
established= 8000 failed= 0 elapsed= 0.2s rate= 37952/s
established= 10000 failed= 0 elapsed= 0.2s rate= 40552/s
established= 12000 failed= 0 elapsed= 0.3s rate= 43569/s
established= 14000 failed= 0 elapsed= 0.3s rate= 47621/s
established= 16000 failed= 0 elapsed= 0.3s rate= 49160/s
established= 18000 failed= 0 elapsed= 0.4s rate= 50737/s
established= 20000 failed= 0 elapsed= 0.4s rate= 53072/s
established= 22000 failed= 0 elapsed= 0.4s rate= 52285/s
established= 24000 failed= 0 elapsed= 0.5s rate= 51673/s
established= 26000 failed= 0 elapsed= 0.5s rate= 53103/s
established= 28000 failed= 0 elapsed= 0.5s rate= 53696/s
established= 30000 failed= 0 elapsed= 0.6s rate= 54280/s
established= 32000 failed= 0 elapsed= 0.6s rate= 55546/s
established= 34000 failed= 0 elapsed= 0.6s rate= 56288/s
established= 36000 failed= 0 elapsed= 0.7s rate= 54817/s
established= 38000 failed= 0 elapsed= 0.8s rate= 50160/s
established= 40000 failed= 0 elapsed= 0.8s rate= 49586/s
established= 42000 failed= 0 elapsed= 0.9s rate= 48847/s
established= 44000 failed= 0 elapsed= 0.9s rate= 46729/s
established= 46000 failed= 0 elapsed= 1.0s rate= 44407/s
established= 48000 failed= 0 elapsed= 1.1s rate= 43836/s
established= 50000 failed= 0 elapsed= 1.2s rate= 43246/s

```

Figure 37: $N = 70\,000$ (target): `ps` + `procstat -f`. RSS=29512 KB (only ~29 MB for 70 000 idle connections), server FDs = 70 009, system-wide FDs = 140 198 / 300 000 (47% of cap).

```

established= 50000 failed= 0 elapsed= 1.2s rate= 43240/s
established= 52000 failed= 0 elapsed= 1.2s rate= 42460/s
established= 54000 failed= 0 elapsed= 1.3s rate= 41736/s
established= 56000 failed= 0 elapsed= 1.4s rate= 41293/s
established= 58000 failed= 0 elapsed= 1.4s rate= 41115/s
established= 60000 failed= 0 elapsed= 1.5s rate= 40416/s
established= 62000 failed= 0 elapsed= 1.5s rate= 40177/s
established= 64000 failed= 0 elapsed= 1.6s rate= 40033/s
established= 66000 failed= 0 elapsed= 1.7s rate= 39810/s
established= 68000 failed= 0 elapsed= 1.7s rate= 39168/s
established= 70000 failed= 0 elapsed= 1.8s rate= 38592/s

[+] connect phase done: established=70000 failed=0 elapsed=1.8s
[+] client PID = 4127
[+] holding 70000 sockets open. Ctrl-C to close all and exit.

While this holds, in another terminal you can run e.g.:
    sockstat -4 -c | wc -l
    netstat -an -p tcp | awk '$6=="ESTABLISHED"' | wc -l
    procstat -f <server-pid> | wc -l
    ps -o pid,rss,vsz,%cpu -p <server-pid>
    netstat -m
    sysctl kern.openfiles kern.maxfiles
[.] holding: sockets_alive=70000 pid=4127 uptime=32s
[.] holding: sockets_alive=70000 pid=4127 uptime=62s

```

Figure 38: $N = 70\,000$: `idle_clients.py` final progress – **70 000 established / 0 failed** with source-IP rotation across `127.0.0.{1..4}` and the `listen(65535)` build.

```

Problems Output Debug Console Terminal Ports
root@ragnarok:/home/harsh/compiled/bonus # ./measure.sh $SPID 5000 | tee run_70k.txt

==== date / uname ====
Wed Sep  9 20:40:32 IST 2026
FreeBSD ragnarok 14.4-RELEASE FreeBSD 14.4-RELEASE releng/14.4-n273675-a456f852d145 GENERIC amd64

==== established connections to server port 5000 ====
ESTABLISHED_to_5000 = 0

==== server open FDs (procstat -f | wc -l) ====
server_open_fds = 70009

==== server memory / cpu (ps) ====
PID  RSS  VSZ  %CPU  %MEM  NLWP  COMMAND
4117 29512 45492  0.1  1.4   1  ./exchange_server 127.0.0.1 5000

==== system-wide open files (sysctl kern.openfiles / kern.maxfiles) ====
kern.openfiles: 140198
kern.maxfiles: 300000
kern.maxfilesperproc: 250000

==== mbuf / socket-buffer usage (netstat -m) ====
6186/1719/7905 mbufs in use (current/cache/total)
2049/1507/3556/500000 mbuf clusters in use (current/cache/total/max)
1/507 mbuf+clusters out of packet secondary zone in use (current/cache)
0/762/762/62503 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/18519 9k jumbo clusters in use (current/cache/total/max)
0/0/0/10417 16k jumbo clusters in use (current/cache/total/max)
5644K/6491K/12136K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 sendfile syscalls
0 sendfile syscalls completed without I/O request
0 requests for I/O initiated by sendfile
0 pages read by sendfile as part of a request
0 pages were valid at time of a sendfile request

```

Figure 39: $N = 70\,000$: `netstat -m` – 2049 mbuf clusters in use, *identical* to the 10k row. Confirms that idle sockets consume zero mbuf clusters at any scale.

```

==== mbuf / socket-buffer usage (netstat -m) ====
6186/1719/7985 mbufs in use (current/cache/total)
2049/1507/3556/500000 mbuf clusters in use (current/cache/total/max)
1/507 mbuf+clusters out of packet secondary zone in use (current/cache)
0/762/762/62503 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/18519 9k jumbo clusters in use (current/cache/total/max)
0/0/0/10417 16k jumbo clusters in use (current/cache/total/max)
5644K/6491K/12136K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 sendfile syscalls
0 sendfile syscalls completed without I/O request
0 requests for I/O initiated by sendfile
0 pages read by sendfile as part of a request
0 pages were valid at time of a sendfile request
0 pages were valid and substituted to bogus page
0 pages were requested for read ahead by applications
0 pages were read ahead by sendfile
0 times sendfile encountered an already busy page

==== socket-buffer sysctls ====
kern.ipc.maxsockbuf: 2097152
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
kern.ipc.somaxconn: 65535

==== server kernel stack (where is it blocked?) ====
  PID   TID COMM      TDNAME      KSTACK
4117 100252 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig_sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_
kevent_generic sys_kevent amd64_syscall fast_syscall_common

==== sockstat summary for server (count) ====
server_sockets_seen_by_sockstat = 70001
root@ragnarok:/home/harsh/compiled/bonus #

```

Figure 40: $N = 70\,000$: `procstat -k + sockstat = 70\,001 (70 000 clients + 1 listener). Server thread still parked at sys_kevent; %CPU 0.1 (tail of the connect flood, not steady-state).`

A.6 Answers to Section 6.9 questions

*Q1. At each connection count, is the server able to maintain all requested connections?
If not, at which count does it first fail?*

Answer. Yes at every measured count. The Exchange Server successfully maintained 10 000 / 20 000 / 30 000 / 40 000 / 50 000 / 60 000 / 70 000 simultaneous idle TCP connections; the established count equals or exceeds the request in every row of Table 2. Two transient RST bursts were observed *during the connect flood* (1 loss at 20 k, 199 losses at 40 k), both caused by the FreeBSD C header `SOMAXCONN=128` capping the listen syncache below what the tuned `kern.ipc.somaxconn=65535` sysctl allowed. After the one-line fix (§A.2) every higher- N run (50 k / 60 k / 70 k) completed with zero failures. No “first-fails” count was reached; the experiment stopped at the assignment’s target of 70 000 with the server still idle at 0 % CPU and ~47 % of the system-wide FD cap unused.

Q2. What is the first significant bottleneck?

Answer. Kernel socket-structure and socket-buffer-cluster pools, not anything under application control. With FreeBSD defaults the first two ceilings hit were:

1. `kern.ipc.maxsockets` ($\approx 64\,177$ on this VM). Loopback consumes two kernel sockets per connection, so the effective ceiling is $\sim 32\,000$ – exactly where `ECONNRESET` first appeared in an untuned exploratory run.
2. `kern.ipc.nmbclusters` ($\approx 125\,007$). The next failure mode would have been `ENOBUFS` on `socket()` itself.

Both are boot-time tunables raised via `/boot/loader.conf` (Table 1). Runtime-tunable ceilings on `kern.maxfiles`, `kern.maxfilesperproc`, `kern.ipc.somaxconn` and `net.inet.ip.portrange.*` were also raised. Once these were lifted, *the server itself* never

became the bottleneck in the 10k–70k range: RSS stayed under 30 MB, %CPU at 0, and `mbuf clusters in use` flat at ~2050 regardless of N (Table 2). The next-ceiling ranking, if scaled further without more tuning, is: `maxsockets` → `nmbclusters` → FD limits → ephemeral ports on the *generator* side (~55k per source IP, addressed by source-IP rotation, §A.3) → reserved socket-buffer memory (`sendspace+recvspace` = 96 KB × 2 sockets/conn).

Q3. How does the concurrency / I/O design contribute to the observed bottleneck?

Answer. It contributes essentially zero. The Exchange Server uses a single-threaded event loop over `kqueue` with non-blocking sockets – the correct primitive on FreeBSD for high-fanout servers. Concretely:

- *One thread, always parked in `kevent()`.* `procstat -k` shows the kernel stack `mi_switch` → `sleepq_wait_sig` → `kqueue_scan` → `sys_kevent` at every N – no per-connection thread stack, no `fd_set` scan (which would cap us at `FD_SETSIZE`), no busy-poll; %CPU is 0 when no events fire.
- *Per-connection application state is ~360 B* (`struct ClientConnection`) plus one pointer slot in `conns[]`; the write buffer is NULL until data is actually queued. RSS scaled at ~390 B/conn across the whole range – dominated by that struct.
- *Connection table doubles.* `conns[]` grows via `realloc` from `MIN_CAPACITY=10`; at $N = 70\,000$ it occupies only ~640 KB.

Because the design has no $O(N)$ work per event and no per-connection resource beyond that small struct, none of the bottlenecks in Q2 can be attributed to it – they are all kernel-side per-connection allocations that any correct TCP server pays.

Q4. Would changing the I/O / concurrency mechanism help with the bottleneck identified?

Answer. No – the current design is already the least-costly alternative. The Q2 bottlenecks (`maxsockets`, `nmbclusters`, `maxfiles`) are kernel-side per-connection allocations paid by any correct TCP server regardless of user-space multiplexing. Concretely:

- *Thread-per-connection* would collapse first: each pthread costs a ~1–2 MB stack, a `struct thread`, and a slot in `kern.threads.max_threads_per_proc` (~1500 by default). At 70k connections it would exhaust threads long before touching the kernel-socket caps we hit.
- *`select()`* caps at `FD_SETSIZE=1024` and performs $O(N)$ scans per event; worse than `kqueue` even at 10k.
- *`poll()`* avoids the `FD_SETSIZE` cap but still scans $O(N)$ on every event; at 70k it would use substantially more CPU than `kqueue`, though it could still scale in principle.

Any alternative therefore either (a) hits the same kernel-resource ceiling while consuming more CPU/memory, or (b) fails earlier for its own reasons. `kqueue` is the right tool; the ceiling we reached is below what `kqueue` is capable of.

Q5. Quantify the trade-off – if you change your implementation or configuration, measure the relevant resource again and compare.

Answer. Exactly one configuration parameter was changed (the `listen()` backlog, §A.2) with all other tuning and the client generator held constant. The relevant resource is “successful

vs. failed connects during the flood”, measured at $N = 40\,000$ before the change and $N = 50\,000$ after (higher load):

Regime	Setting	Established / requested	Lost
Before	<code>listen(server_fd, SOMAXCONN), SOMAXCONN=128</code>	39 797 / 40 000	199 (0.50 %)
After	<code>listen(server_fd, 65535)</code>	50 000 / 50 000	0 (0.00 %)

Higher load, same tuning, same generator settings, same measurement recipe – the one-line change eliminated the losses entirely, and 60 k and 70 k subsequently completed with 0 failures as well (Table 2). **Runtime cost of the change: zero.** Raising the backlog does not allocate anything eagerly; it only sets the cap on how many half-open connections the syncache is allowed to hold. No change to accepted-connection memory, CPU, latency, or protocol behaviour.

A.7 Notes and caveats

- **Loopback doubles the kernel-socket cost.** Every connection in this experiment holds two kernel sockets (client end + accepted server end) because both live in the same VM. A real deployment driven from a second host would need roughly half the `kern.ipc.maxsockets` headroom for the same N .
- **The 40 k row is intentionally left in the table with its 199 losses.** It is the direct evidence for the `SOMAXCONN` bug and the A/B pair for Q5. Re-running it on the fixed build would also yield 0 losses (as 50 k / 60 k / 70 k already show).
- **Idle vs. active is a very different workload.** All rows above measure *idle* connections, per the handout. Active traffic would allocate mbuf clusters proportional to in-flight bytes; `netstat -m` was flat here only because no data flowed. The order-book code path itself was exercised in Experiments 2–8; scaling that under load is out of scope for Section 6.9.
- **Non-zero %CPU at 60 k and 70 k** (0.2 % and 0.1 %) is the tail of the connect flood being sampled before it fully subsided, not a steady-state cost of holding the connections. `procstat -k` confirmed the server was already back in `sys_kevent` at snapshot time.
- **Reproducibility.** Every command and setting used to produce Table 2 is listed in `compiled/bonus/README.m`. per-run raw outputs are stored as `compiled/bonus/{10k,20k,...,70k}/*.txt` in the submission alongside the screenshots reproduced in this appendix.